Message Authentication and Hash Functions

Unit III

Rolisha Sthapit

Overview

- Message Authentication
- Hash Functions
- Message Digests: MD4 and MD5
- Secure Hash Algorithms: SHA-1,SHA-2
- HMAC
- Digital Signature

Textbook:

Cryptography and Network Security, William Stallings

Part Three: Cryptographic Data Integrity Algorithms

Message Authentication

- Message authentication is a mechanism or service used to verify the **integrity** and **authenticity** of a received or stored message. Message authentication assures that data received are exactly as sent, providing assurance to the receiver that the message has not been altered and is indeed from the purported sender.
- A message, file, document, or other collection of data is said to be **authentic** when it is **genuine** and came from its **alleged source**.
- The two important aspects are to verify:
  - that the contents of the message have **not been altered**, and
  - that the source is **authentic**.
- We may also wish to verify a message's
  - **timeliness** (not artificially delayed and replayed), and
  - **sequence** relative to other messages flowing between two parties.

Message Authentication Schemes

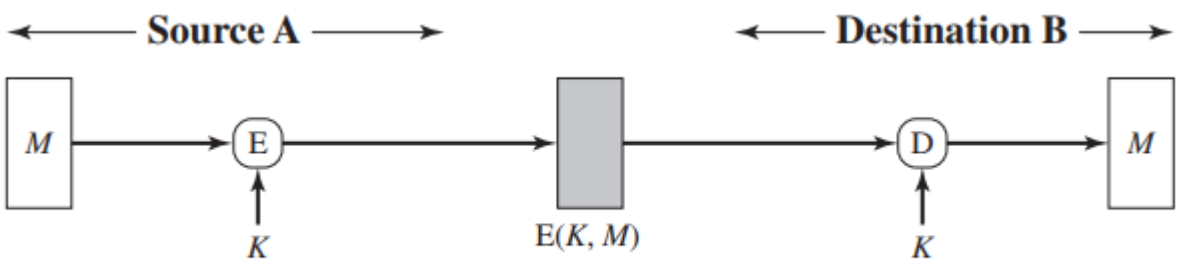
- Message encryption: The ciphertext of the entire message serves as its authenticator.
- Message authentication code (MAC): A function of the message and a secret key that produces a fixed-length value (MAC) that serves as the authenticator.
- Hash function: A function that maps a message of any length into a fixed length value (hash value or message digest), which serves as the authenticator.
- Hash function-based MAC (HMAC)
- Digital Signature: The signature is formed by taking the hash of the message and encrypting the message with the creator's private key. The signature serves as the authenticator.

Authentication Using Symmetric Encryption

- Encryption ensures confidentiality and Authentication ensures authenticity and integrity.
- Authentication using symmetric encryption involves ensuring that only the intended sender and receiver, who share a secret key, can communicate securely and verify the authenticity of the messages.
- Because only the sender and receiver share the secret key, the sender can encrypt the message in a way that only the receiver can decrypt and verify.
- Symmetric encryption alone is not sufficient for robust data authentication due to certain vulnerabilities such as block reordering attacks (each block will decrypt; the reordering may alter the meaning of the overall data sequence)

Authentication Using Symmetric Encryption

- Combining with other modes of operation and techniques can provide robust authentication and integrity guarantees:
- Including an error-detection code in the message ensures that any alterations to the message can be detected and integrity is maintained.
- Including a sequence number in the message helps ensure that messages are received in the correct order and not replayed by an attacker.
- Including a timestamp in the message helps ensure that messages are not delayed excessively, which can be a sign of replay attacks.



(a) Symmetric encryption: confidentiality and authentication

Authentication without Message Encryption

- Encryption ensures confidentiality and Authentication ensures authenticity and integrity.
- Encryption and authentication: confidentiality and authenticity.
- Situations where authentication without confidentiality is preferred:
  - **In broadcast scenarios** (notifications), a single plaintext message with an authentication tag ensures that all recipients can verify the message's authenticity.
  - **Under heavy loads** (in an exchange), systems cannot afford the time to decrypt all incoming messages. Selective authentication of plaintext is done randomly.
  - **For executable programs**, plaintext distribution with authentication tags allows for efficient execution and periodic integrity checks, ensuring the program remains unaltered.
- For applications where confidentiality is not a primary concern, these approaches leverage the balance between security (through authentication) and operational efficiency (by avoiding the overhead of decryption).

Message Authentication Code

- A number of algorithms could be used to generate the MAC.
- The NIST specification FIPS PUB 113 recommends the use of **DES (Data Encryption Standard)** for generating a MAC.
  - DES is used to encrypt the message M.
  - The last number of bits (16 or 32) of the ciphertext are used as the MAC.
  - The message M and MAC are appended and transmitted.

- **Encryption vs. Authentication:**
  - The authentication algorithm does not need to be reversible, unlike encryption which must be reversible for decryption.
  - Because of this, the authentication function has mathematical properties that make it less vulnerable to certain types of attacks compared to encryption functions.

Message Authentication Code (MAC)

- One common technique for message authentication involves the use of a secret key to generate a Message Authentication Code (MAC).
- **Secret Key Sharing**: Two parties, A and B, share a common secret key K_{AB}
- **Message Preparation**: When party A wants to send a message M to party B, it calculates the MAC using a complex function F that involves the secret key and the message: $\mathrm{MAC}_M = F(K_{AB}, M)$
- **Message Transmission**: The message M along with the MAC MAC_M is sent to the recipient B.
- **MAC Verification**: Upon receiving the message, party B performs the same calculation using the shared secret key K_{AB} to generate a new MAC from the received message: $\mathrm{MAC}_{M'} = F(K_{AB}, M')$. B compares the received MAC MAC_M with the calculated MAC $\mathrm{MAC}_{M'}$. If they match, the message is authenticated.

Message Authentication Code

Assurance Provided by MAC

1. Message Integrity:

- The receiver is assured that the message has not been altered. As attacker does not know the secret key, they cannot generate a correct MAC for the altered message. If an attacker alters the message without altering the MAC, the MAC calculated by the receiver will differ from the received MAC, indicating a tampered message.

2. Message Authenticity:

- The receiver is assured that the message is from the alleged sender because only someone with the secret key can generate the correct MAC. This ensures that the message is indeed from the legitimate sender, A.

3. Proper Sequencing:

- If the message includes a sequence number, the receiver can ensure the proper order of messages. An attacker cannot alter the sequence number without being detected, as they cannot generate a valid MAC for the altered message.

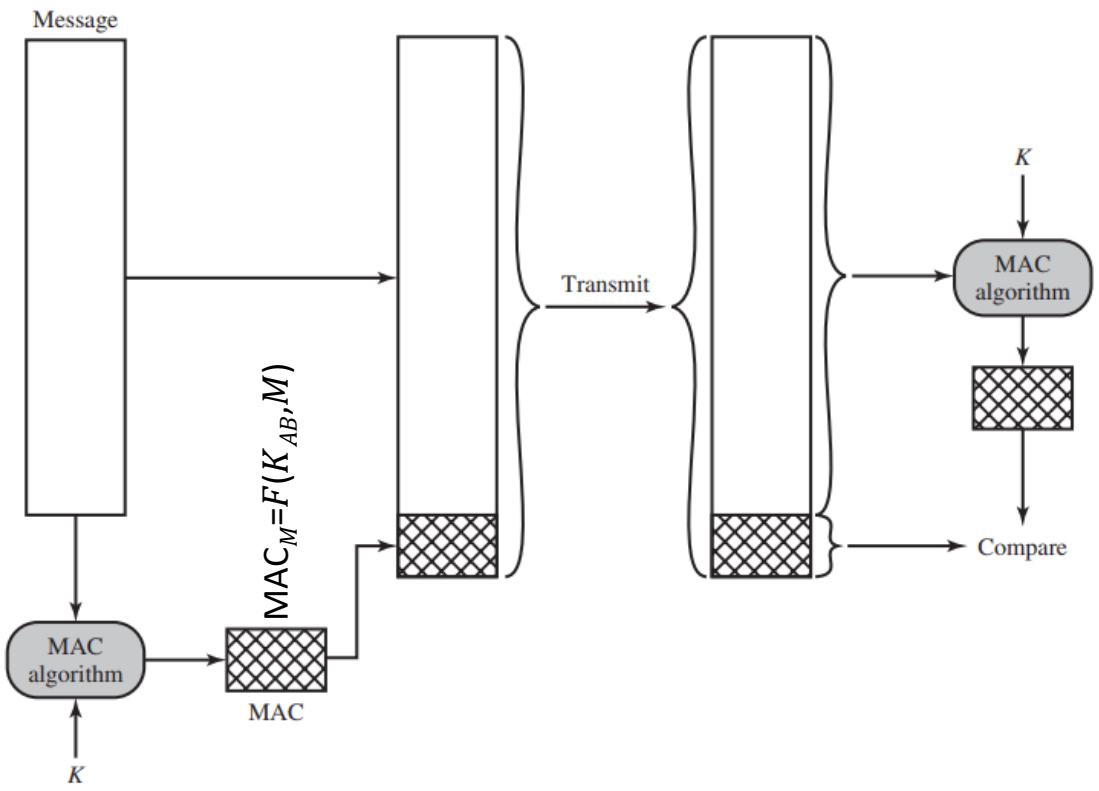
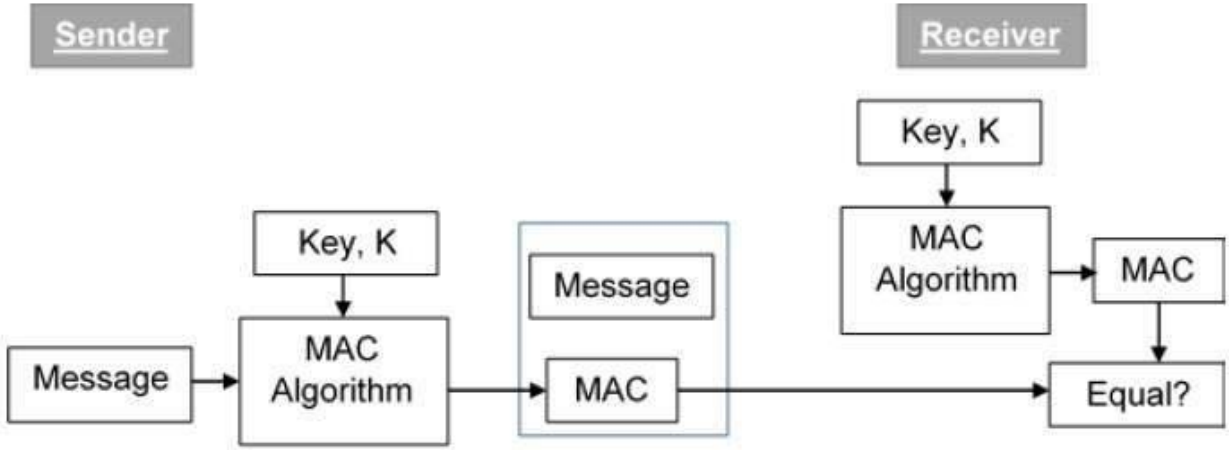


Figure 2.3 **Message Authentication Using a Message Authentication Code (MAC)**

Message Authentication Code

- MAC algorithm is a symmetric key cryptographic technique to provide message authentication. For establishing MAC process, the sender and receiver share a symmetric key K.
- Essentially, a MAC is an encrypted checksum generated on the underlying message that is sent along with a message to ensure message authentication.
- The process of using MAC for authentication is depicted in the following illustration:



Message Authentication Code

1. The sender uses some publicly known MAC algorithm, inputs the message and the secret key K and produces a MAC value.
2. Similar to hash, MAC function also compresses an arbitrary long input into a fixed length output. The major difference between hash and MAC is that MAC uses secret key during the compression.
3. The sender forwards the message along with the MAC. Here, we assume that the message is sent in the clear, as we are concerned of providing message origin authentication, not confidentiality. If confidentiality is required then the message needs encryption.
4. On receipt of the message and the MAC, the receiver feeds the received message and the shared secret key K into the MAC algorithm and re-computes the MAC value.
5. The receiver now checks equality of freshly computed MAC with the MAC received from the sender. If they match, then the receiver accepts the message and assures himself that the message has been sent by the intended sender.
6. If the computed MAC does not match the MAC sent by the sender, the receiver cannot determine whether it is the message that has been altered or it is the origin that has been falsified. As a bottom-line, a receiver safely assumes that the message is not the genuine.

Properties of MAC

- If an opponent observes M and MAC_M, it should be computationally infeasible for the opponent to construct a message M' such that $MAC_M = MAC_{M'}$
- MAC_M should be uniformly distributed: For randomly chosen messages M and M', the probability that $MAC_M = MAC_{M'}$ should be very low (2^{-n} where n is the number of bits in the MAC).
- Let M' be generated by changing one or more bits of M. In that case, the probability that $MAC_M = MAC_{M'}$ should be very low (2^{-n}).

- Suppose that we are using 100-bit messages and a 10-bit MAC. Then, there are a total of 2^{100}different messages but only 2^{10} different MACs. So multiple messages can have same MAC. This is known as collision. Using more bits per MAC reduces the chances of collision.

Hash Functions

- A hash function is a cryptographic tool used for ensuring data integrity and authenticity.
- Unlike the Message Authentication Code (MAC) that requires a secret key shared between the sender and receiver, a hash function takes an arbitrary block of data M and returns a fixed-size string of bytes $h = \mathrm{H}(M)$. This output is typically called the **hash value** or **message digest**.
- A good hash function has the property that the results of applying the function to a large set of inputs will produce outputs that are evenly distributed and apparently random.
- In general terms, the principal objective of a hash function is data integrity. A change to any bit or bits results, with high probability, in a change to the hash value.

Hash Function

- Figures depict the general operation of a cryptographic hash function.
- Typically, the message is padded out to an integer multiple of some fixed length (e.g., 512 bits, 1024 bits) and the padding includes the value of the length of the original message in bits.
- The length field is a security measure to increase the difficulty for an attacker to produce an alternative message with the same hash value.

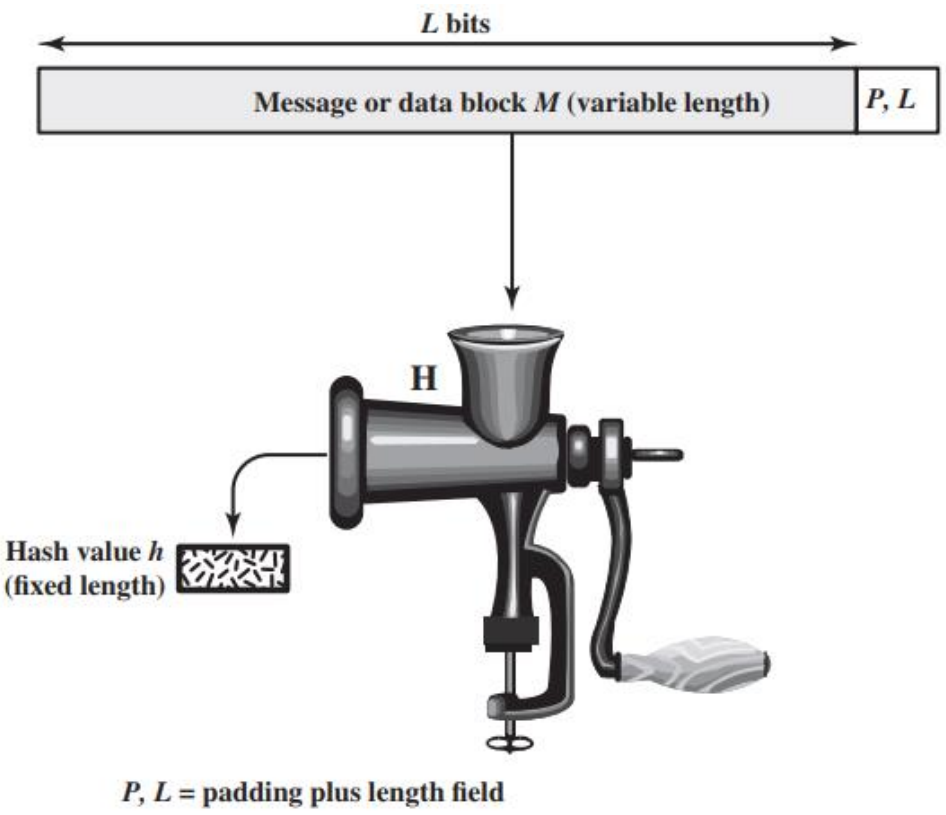


Figure 2.4 Cryptographic Hash Function; $h = H(M)$

Hash Function

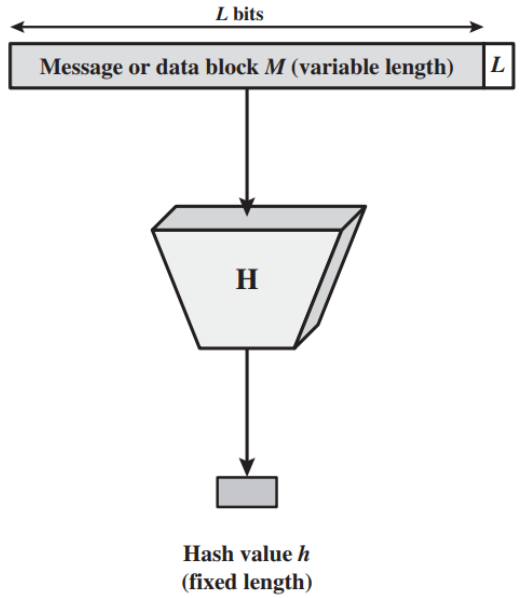


Figure 11.1 Black Diagram of Cryptographic Hash Function; $h = \mathrm{H}(M)$

- **Input and Output**:
  - Input: An arbitrary length message M.
  - Output: A fixed-size hash value h (e.g., 256 bits for SHA-256).

    i.e., $h = \mathrm{H}(M)$ where H is the Hash function.
- **Properties of Hash Functions**:
  - Deterministic: The same input will always produce the same output.
  - Pre-image Resistance: Given a hash value h, it should be computationally infeasible to find any message M such that $h=\mathrm{H}(M)$.
  - Second Pre-image Resistance: Given an input M_1, it should be computationally infeasible to find another input M_2 such that $\mathrm{H}(M_1)=\mathrm{H}(M_2)$
  - Collision Resistance: It should be computationally infeasible to find any two distinct messages M_1 and M_2 such that $\mathrm{H}(M_1)=\mathrm{H}(M_2)$.

Hash Function

- **Example**
- Let's say we have a hash function HHH and it produces a hash value hhh from an input message MMM:

1. You put the word "hello" (input message MMM) into the hash function HHH.
2. The hash function HHH turns "hello" into a complex string of characters, like "5d41402abc4b2a76b9719d911017c592" (hash value hhh).

- **Pre-image resistance** means that if you only have "5d41402abc4b2a76b9719d911017c592", you shouldn't be able to work backward to figure out that the original message was "hello".
- This property is important because it keeps information secure. For example, if a password is stored as a hash value, even if someone gets hold of the hash value, they shouldn't be able to figure out the original password easily.

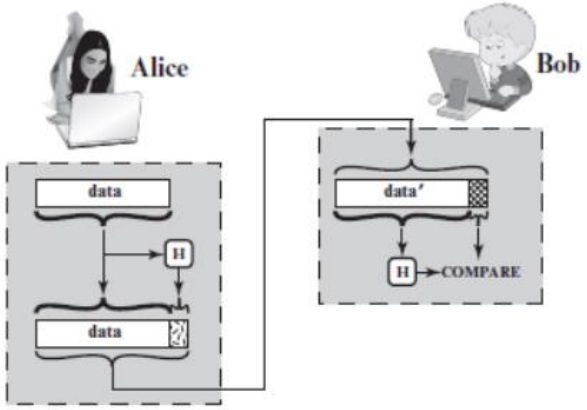
Hash Function

- Hash functions are also called **one way Hash Function**.
- It means the function works in forward direction only, no inverse exists.

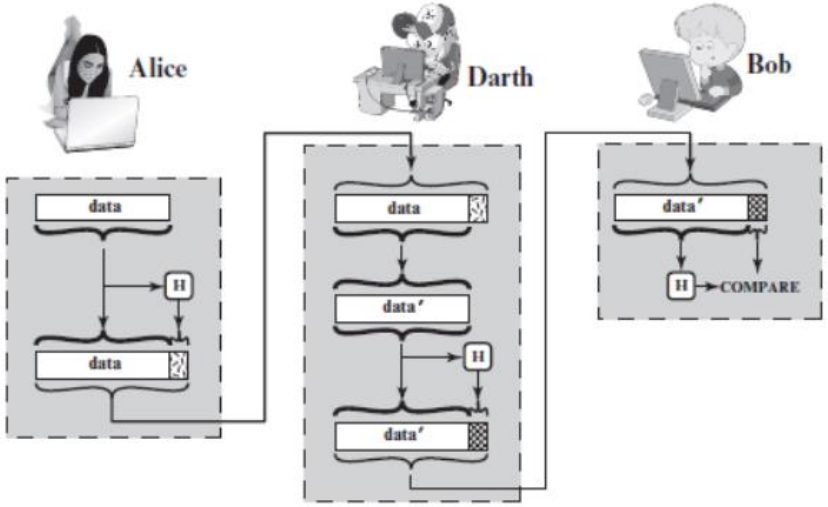
  - Computationally infeasible to find the input from the output (pre-image resistance)
  - Easy to compute but hard to invert (irreversibility)
  - A small change in the input message results in a significantly different hash value (Avalanche effect)

Essence of the use of Hash Function

- **figure (a)**
- The sender computes a hash value as a function of the bits in the message and transmits both the hash value and the message.
- The receiver performs the same hash calculation on the message bits and compares this value with the incoming hash value.
- If there is a mismatch, the receiver knows that the message (or possibly the hash value) has been altered.



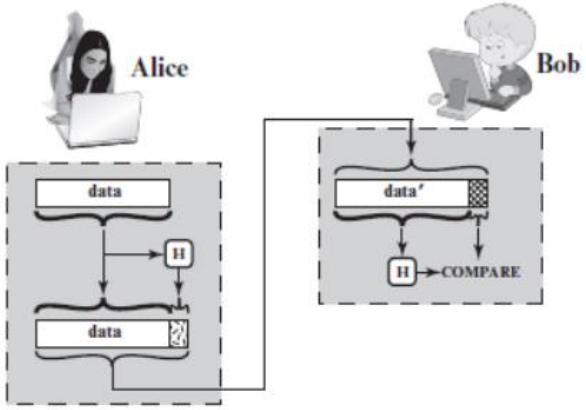
(a) Use of hash function to check data integrity



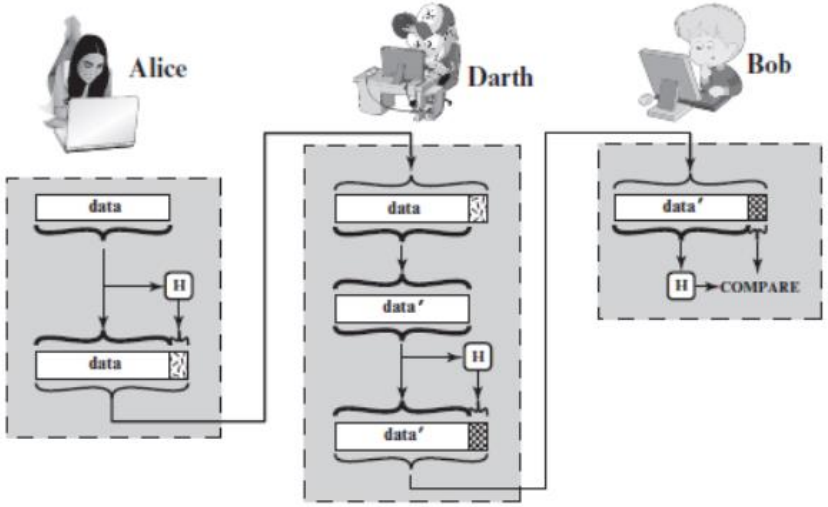
(b) Man-in-the-middle attack

Essence of the use of Hash Function

- **figure (b)**
- The hash value must be transmitted in a secure fashion. That is, the hash value must be protected so that if an adversary alters or replaces the message, it is not feasible for adversary to also alter the hash value to fool the receiver.
- In this example, Alice transmits a data block and attaches a hash value. Darth intercepts the message, alters or replaces the data block, and calculates and attaches a new hash value. Bob receives the altered data with the new hash value and does not detect the change. To prevent this attack, the hash value generated by Alice must be protected.



(a) Use of hash function to check data integrity



(b) Man-in-the-middle attack

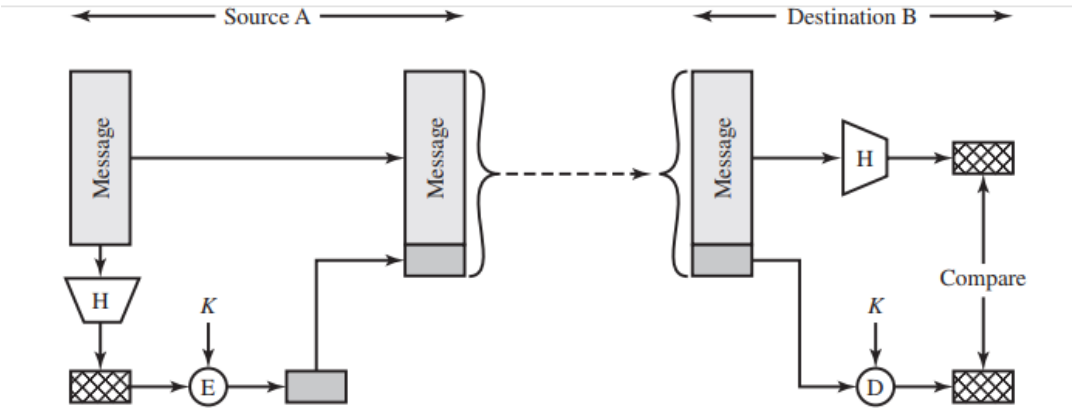
Applications of Hash Functions

- Message Authentication: (next pages)
- Digital Signature: the hash value of a message is encrypted with a user's private key. Anyone who knows the user's public key can verify the integrity of the message that is associated with the digital signature.
- Passwords Storage: A hash of a password is stored by an operating system rather than the password itself.
- Intrusion/ Virus Detection: Store hash value for each file on a system. One can later determine if a file has been modified by comparing the hash value.

Applications of Hash Functions

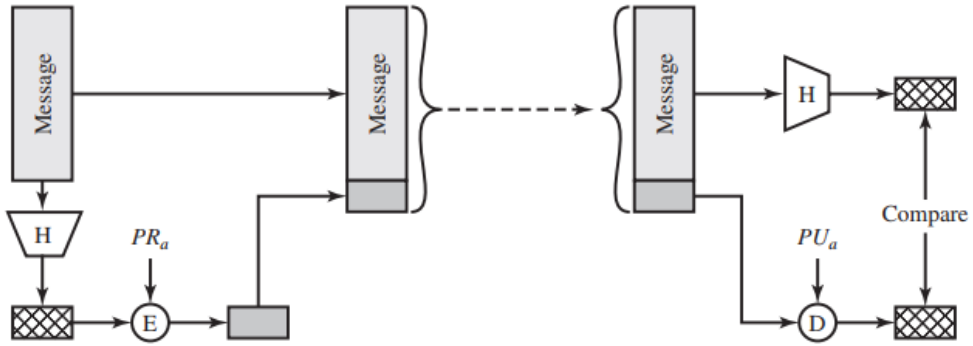
Only the Hash Value is Encrypted.

- Message Authentication:
- (a) The message digest can be encrypted using symmetric encryption.
- If it is assumed that only the sender and receiver share the encryption key, then authenticity is assured.



(a) Using symmetric encryption

- (b) The message digest can also be encrypted using public-key encryption. It provides a digital signature as well as message authentication.
- And it does not require the distribution of keys to communicating parties.



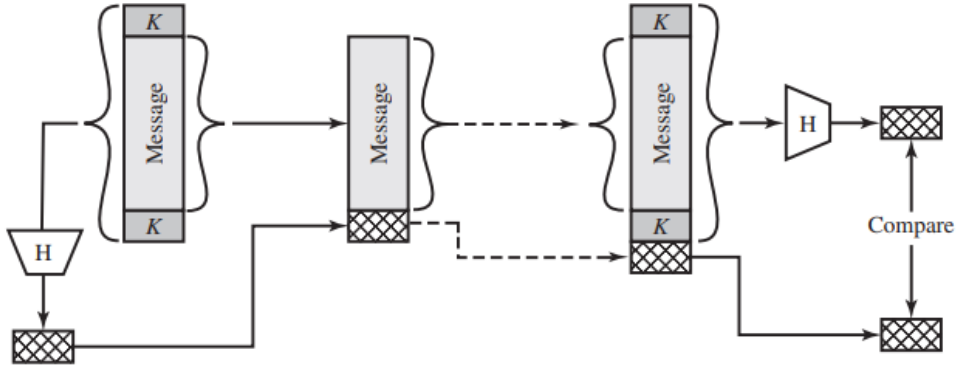
(b) Using public-key encryption

- Encryption systems incur additional cost and slower the process.

Applications of Hash Functions

No Encryption is used.

- Message Authentication:
- (c) The message digest is generated by incorporating the secret key but no encryption is used.
- As only sender and receiver know the secret key, receiver can verify the integrity.
- This technique is known as a keyed hash MAC.



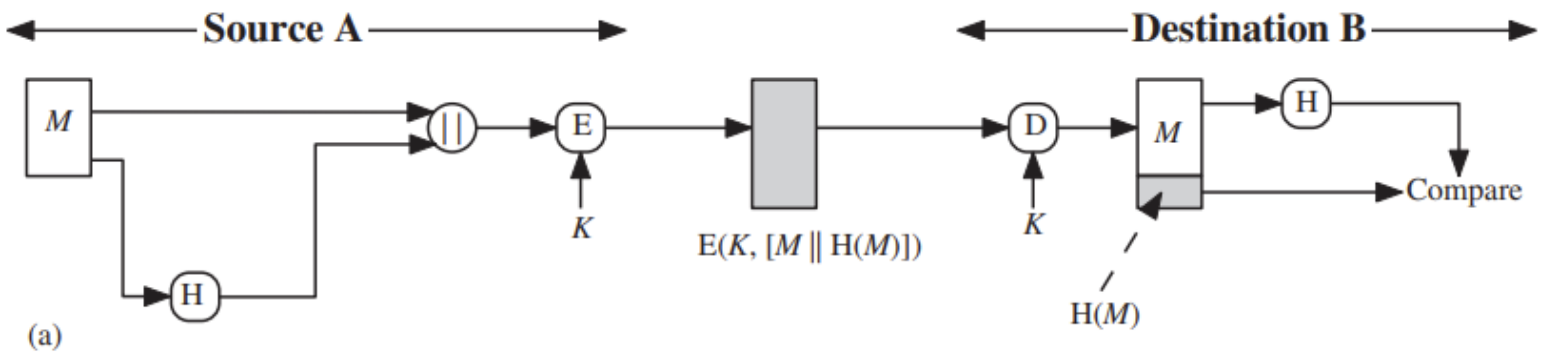
(c) Using secret value

Figure 2.5 **Message Authentication Using a One-Way Hash Function**

Applications of Hash Functions

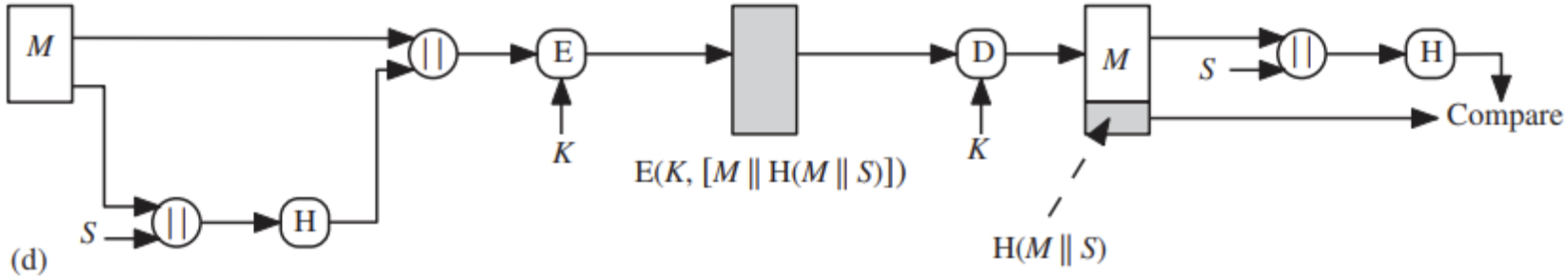
Message and Hash Value both are Encrypted.

- Message Authentication



- (a) The message plus concatenated hash code is encrypted using symmetric encryption.

Encryption systems incur additional cost, slower the process and are optimized mainly for large data sizes. Encryption of whole message and hash value are done only if authentication as well as confidentiality are essential.



- (b) The message plus concatenated hash code (generated using common Secret Key K) is encrypted using symmetric encryption.

Simple Hash Function

- All hash functions operate using the following general principles:
- The input (message, file, etc.) is viewed as a sequence of n-bit blocks. The input is processed one block at a time in an iterative fashion to produce an n-bit hash function.
- One of the simplest hash functions is the bit-by-bit XOR of every block. This can be expressed as follows:
- $C_i = b_{i1} + b_{i2} + b_{i3} + \ldots \ldots \ldots + b_{im}$

  where,

  C_i = i^{th} bit of hash code,

  $1 <= i <= n$,

  n = number of bits in a data block

  b_{im} = i^{th} bit of mth block.

	Bit 1	Bit 2		Bit n
Block 1	b_{11}	b_{21}		b_{n1}
Block 2	b_{12}	b_{22}		b_{n2}
Block m	b_{1m}	b_{2m}		b_{nm}
Hash Code	C_1	C_2		C_n

Message Digest

- A message digest is a fixed-size numerical representation (usually a string of characters or bytes) of a message or piece of data, **produced by a hash function**.
- It serves as a unique fingerprint of the data, ensuring that even a small change in the input will result in a significantly different digest.
- Message digests are used in various cryptographic applications to ensure data integrity and authenticity.
- Common examples of hash algorithms used to generate MAC values include **MD4**, **MD5** and **Secure Hash Algorithms**.
- **MD4** and **MD5** are cryptographic hash functions designed by Ronald Rivest. They take an input (or message) and produce a fixed-size hash value (**message digest**), typically represented as a hexadecimal number.

Message Digest

- Adding a unique symmetric key, that is shared between a sender and receiver in order to compute a message digest value provides confidentiality to ensure that the message digest cannot be easily changed if the data is changed in an unauthorized or other unexpected manner. Both the sender and receiver of the data (including the sender's message digest) must share the same key for the receiver to generate an identical message digest.
- If some other agent changes the data between the sender and receiver and hashes their own message digest without the correct key, the new message digest, while representing the data, will not be the same as the message digest computed by the receiver using the correct symmetric key. The resulting value generated from the symmetric key (or MAC key) and the message digest is known as a Message Authentication Code (MAC) because it can be used to test for unauthorized alteration of both the data and the message digest.

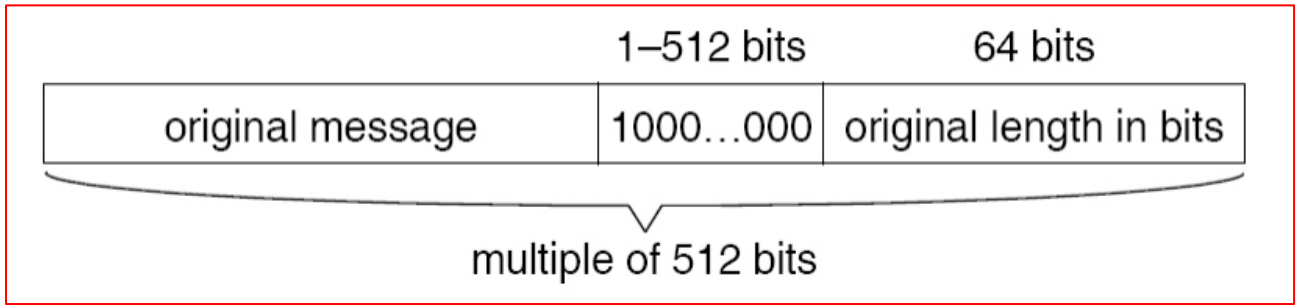
MD4 (Message Digest Algorithm 4)

- The MD4 Message-Digest Algorithm is a cryptographic hash function developed by Ronald Rivest in 1990. The digest length is 128 bits. The algorithm has influenced later designs, such as the MD5, SHA-1 algorithms.
- The security of MD4 has been severely compromised. The first full collision attack against MD4 was published in 1995, and several newer attacks have been published since then.

https://www.ietf.org/rfc/rfc1320.txt

MD4 (Message Digest Algorithm 4)

- 128-bit hash using 512-bit blocks using 32-bit operations
- **Step 1:** Let M be the message to hash
- **Step 2: Padding:**



- The original message is padded to ensure its length is congruent to 448 modulo 512.
- Padding is done by appending a single '1' bit followed by '0' bits until the message length is 64 bits of a multiple of 512 bits.
- The length of the original message (before padding) is appended as a 64-bit integer.
- Padding is always performed, even if the length of the message is already congruent to 448, modulo 512

m' = m100 … 0
- Pad m until it is 64 bits short of a multiple of 512
- Message is always padded (i.e. even 448 bits are padded)
- Append a 1 followed by 0's :

M[0 ... N-1] = m' with low order 64 bits(indicating length of message before padding) of m appended to it
- N is a multiple of 16

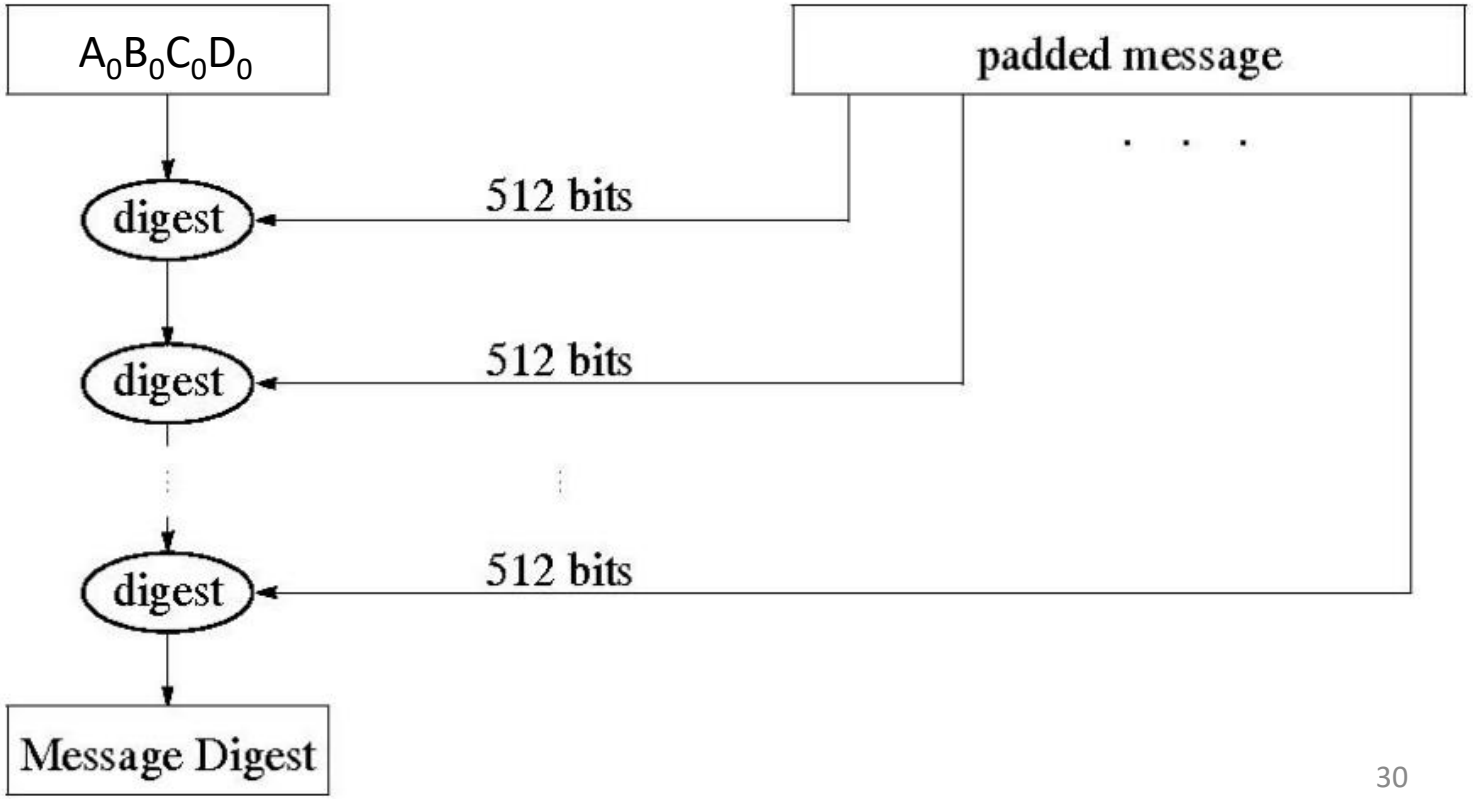
https://www.ietf.org/rfc/rfc1320.txt

MD4

- **Step 3:** Divide the 512 bits message blocks in to different words with 32 bits each $\mathbf{M_0, M_1, \ldots\ldots\ldots M_{15}}$

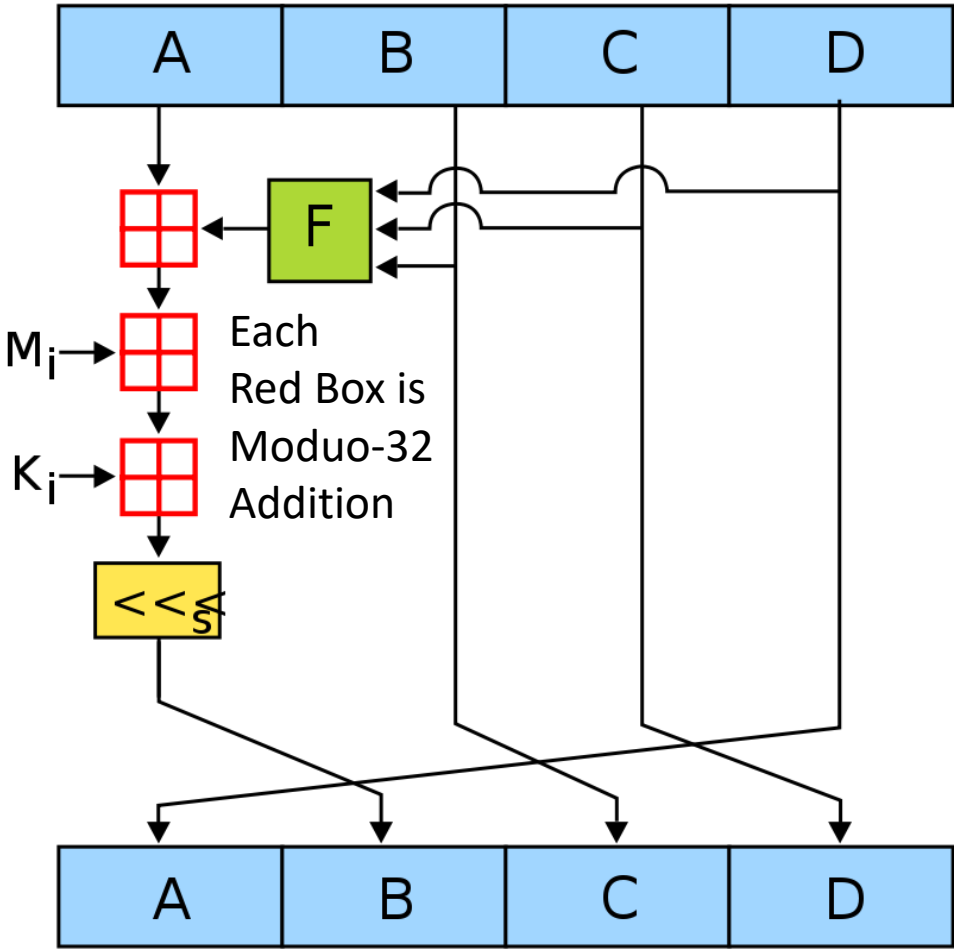
- **Step 4: Initialization:**
- A four-word buffer (A,B,C,D) is used to compute the message digest. Here each of A, B, C, D is a 32-bit register and each are initialized to specific values (128 bits)

- word A: 01 23 45 67
- word B: 89 ab cd ef
- word C: fe dc ba 98
- word D: 76 54 32 10
- Also denoted as $A_0B_0C_0D_0$



MD4

- **<u>Step 5: Processing:</u>**
- The padded message is divided into 512-bit blocks. Each block is subdivided into 16 words of 32-bit each (M_0 to M_{15}).
- Each block undergoes three groups of processing, each group consisting of 16 round operations. Total Perform 48 rounds of operations
- <u>For round 0-15</u>
- F(B,C,D) = BC OR [NOT(B)D]
- i = 0 to 15 (for M_i)
- $K_i = 0$
- s = 3, 7, 11, 19 bits left rotations (4 times)



https://people.computing.clemson.edu/~jmarty/courses/commonCourseContent/AdvancedModule-SecurityConceptsAndApplicationToLinux/the_md_message_digest_algorithm-MD4-Rivest-1998.pdf

MD4

- For round 16-31
- F(B,C,D) = BC OR BD OR CD
- i = 0, 4, 8, 12, 1, 5, 9, 13, 2, 6, 10, 14, 3, 7, 11, 15 (for M_i)
- K_i = 5A827999
- s = 3, 5, 9, 13 bits left rotations (4 times)
- For round 32-47
- F(B,C,D) = B XOR C XOR D
- i = 0, 8, 4, 12, 2, 10, 6, 14, 1, 9, 5, 13, 3, 11, 7, 15 (for M_i)
- K_i = 6ED9EBA1
- s = 3, 9, 11, 15 bits left rotations (4 times)

- F(X,Y,Z) = XY v not(X) Z
  - Bitwise conditional: if X then Y else Z.
- G(X,Y,Z) = XY v XZ v YZ
  - Bitwise majority function: bit positions in which 2 or more bits are 1, output has a 1, else output has a 0
- H(X,Y,Z) = X ⊕ Y ⊕ Z
  - Bit positions with odd number of 1's are 1, rest are 0

where ∧, ∨, ¬, ⊕ are AND, OR, NOT, XOR

- **Step 6: Compute:**
- The final values of the registers A, B, C, and D are added with the initial values and concatenated to produce the 128-bit hash value. $A = A + A_0$, $B = B + B_0$, $C = C + C_0$ $D = D + D_0$

MD4

- Final value of A, B, C, D is the message digest for first block of message. Perform this operation for each blocks of message. For each new blocks, the output A, B, C, D from the last block serve as the initial value. And the final message digest is the result of MD4 algorithm.

- The security of MD4 has been severely compromised.
- The first full collision attack against MD4 was published in 1995, and several newer attacks have been published since then.
- The algorithm has influenced later designs, such as the MD5, SHA-1 algorithms.

MD4

Do the following:

```
For i = 0 to N/16-1 do        /* process each 16-word block */
        Set X[j] to M[i*16+j], for j = 0, 1, ..., 15.
        Save A as AA, B as BB, C as CC, and D as DD.

        [Round 1]
          Let [A B C D i s] denote the operation
                A = (A + f(B,C,D) + X[i]) <<< s .
          Do the following 16 operations:
                [A B C D 0  3]
                [D A B C 1  7]
                [C D A B 2  11]
                [B C D A 3  19]
                [A B C D 4  3]
                [D A B C 5  7]
                [C D A B 6  11]
                [B C D A 7  19]
                [A B C D 8  3]
                [D A B C 9  7]
                [C D A B 10 11]
                [B C D A 11 19]
                [A B C D 12 3]
                [D A B C 13 7]
                [C D A B 14 11]
                [B C D A 15 19]
```

```
[Round 2]
  Let [A B C D i s] denote the operation
        A = (A + g(B,C,D) + X[i] + 5A827999) <<< s .
  Do the following 16 operations:
        [A B C D 0  3]
        [D A B C 4  5]
        [C D A B 8  9]
        [B C D A 12 13]
        [A B C D 1  3]

        [D A B C 5  5]
        [C D A B 9  9]
        [B C D A 13 13]
        [A B C D 2  3]
        [D A B C 6  5]
        [C D A B 10 9]
        [B C D A 14 13]
        [A B C D 3  3]
        [D A B C 7  5]
        [C D A B 11 9]
        [B C D A 15 13]
```

	Octal	Hex
Round 2 constant ($\sqrt{2}$):	013240474631	5A827999
Round 3 constant ($\sqrt{3}$):	015666365641	6ED9EBA1

MD4

```
[Round 3]
  Let [A B C D i s] denote the operation
        A = (A + h(B,C,D) + X[i] + 6ED9EBA1) <<< s .
  Do the following 16 operations:
        [A B C D 0  3]
        [D A B C 8  9]
        [C D A B 4  11]
        [B C D A 12 15]
        [A B C D 2  3]
        [D A B C 10 9]
        [C D A B 6  11]
        [B C D A 14 15]
        [A B C D 1  3]
        [D A B C 9  9]
        [C D A B 5  11]
        [B C D A 13 15]
        [A B C D 3  3]
        [D A B C 11 9]
        [C D A B 7  11]
        [B C D A 15 15]
```

MD4 Round 1

Function operates on chaining variable, adds in message block

/* [abcd k s] denotes a = (a + F(b,c,d) + X[k]) <<< s. */
/* 16 operations. */
[ABCD 0 3]; [DABC 1 7]; [CDAB 2 11] ; [BCDA 3 19];
[ABCD 4 3]; [DABC 5 7] ; [CDAB 6 11]; [BCDA 7 19];
[ABCD 8 3]; [DABC 9 7]; [CDAB 10 11]; [BCDA 11 19]
[ABCD 12 3]; [DABC 13 7]; [CDAB 14 11]; [BCDA 15 19];

Note: each word rotates through each of the four positions for each value of s
X[k] (M) combined with A,B,C,D

Words sequential in round 1 (i.e. k = 1,2,3,…. 15 in order)

MD4 Round 2

```
/* [abcd k s] denotes
   a = (a + G(b,c,d) + X[k] + 0x5A827999) <<< s. */
/* 16 operations. */
[ABCD 0 3]; [DABC 4 5]; [CDAB 8 9]; [BCDA 12 13];
[ABCD 1 3]; [DABC 5 5]; [CDAB 9 9]; [BCDA 13 13];
[ABCD 2 3]; [DABC 6 5]; [CDAB 10 9]; [BCDA 14 13];
[ABCD 3 3]; [DABC 7 5]; [CDAB 11 9]; [BCDA 15 13];
```

Word ordering altered from round 1

MD4 Round 3

```
/* Let [abcd k s] denotes
   a = (a + H(b,c,d) + X[k] + 0x6ED9EBA1) <<< s. */
/* 16 operations. */
[ABCD 0 3]; [DABC 8 9]; [CDAB 4 11]; [BCDA 12 15]
[ABCD 2 3]; [DABC 10 9]; [CDAB 6 11]; [BCDA 14 15]
[ABCD 1 3]; [DABC 9 9]; [CDAB 5 11]; [BCDA 13 15];
 [ABCD 3 3]; [DABC 11 9]; [CDAB 7 11]; [BCDA 15 15]
```

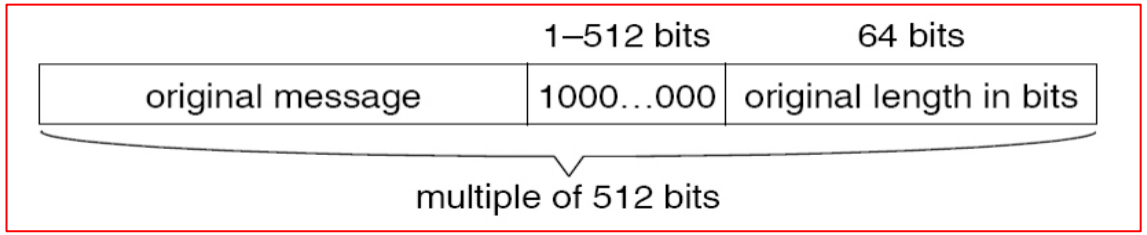
Word ordering partially altered from round 2

MD5 (Message Digest Algorithm 5)

- The MD5 (message-digest algorithm) hashing algorithm is a one-way cryptographic function that accepts a message of any length as input and returns as output a fixed-length digest value to be used for authenticating the original message.
- The MD5 hash function was originally designed for use as a secure cryptographic hash algorithm for authenticating digital signatures.
- MD5 also produces a 128-bit message digest.
- MD5 hashing is no longer considered reliable for use as a cryptographic checksum because security experts have demonstrated techniques capable of easily producing MD5 collisions on commercial off-the-shelf computers.

MD5

- 128-bit hash using 512-bit blocks using 32-bit operations
- **Step 1**: Let M be the message to hash
- **Step 2: Padding:**



- The original message is padded to ensure its length is congruent to 448 modulo 512.
- Padding is done by appending a single '1' bit followed by '0' bits until the message length is 64 bits shy of a multiple of 512 bits.
- The length of the original message (before padding) is appended as a 64-bit integer.
- Padding is always performed, even if the length of the message is already congruent to 448, modulo 512

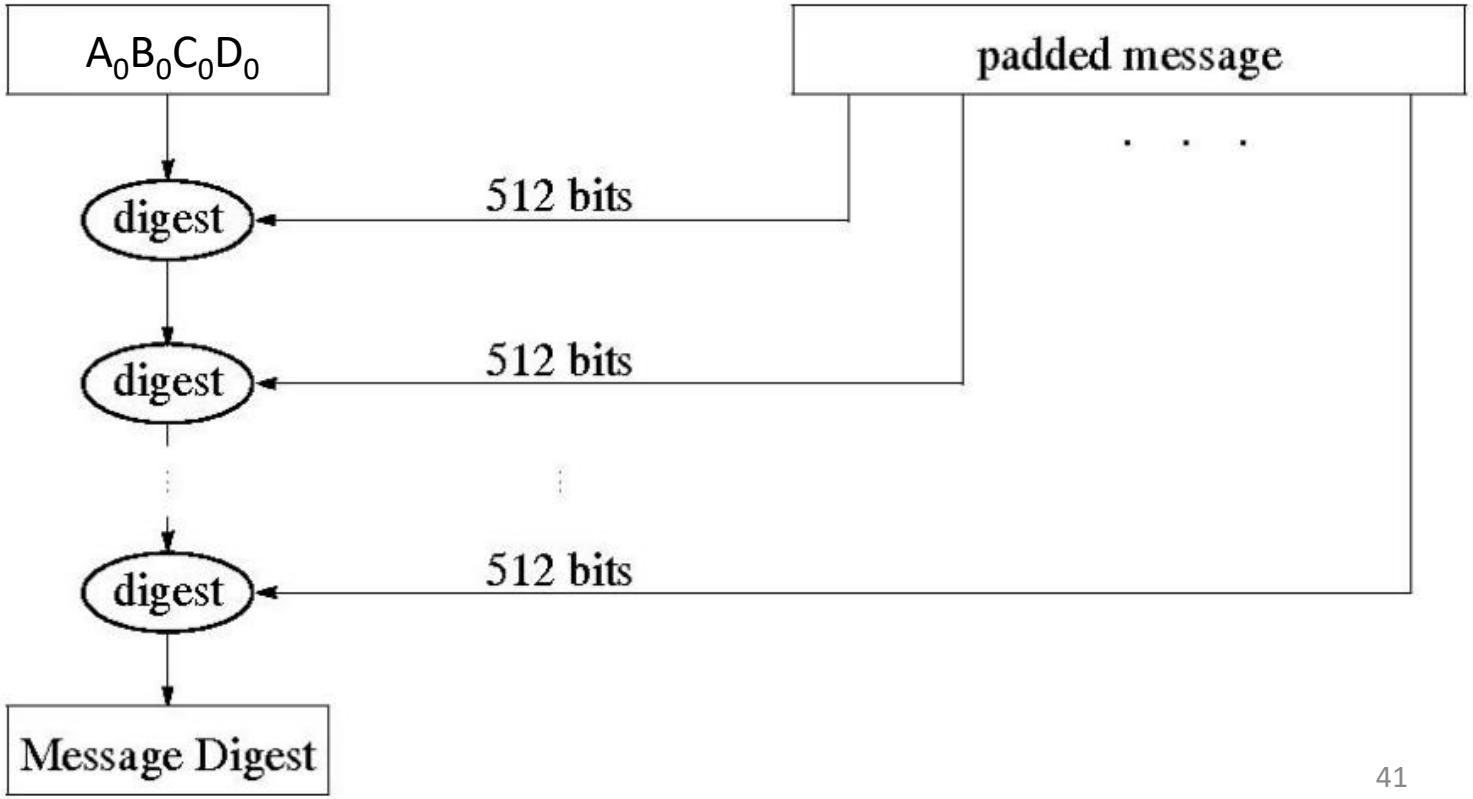
https://www.ietf.org/rfc/rfc1320.txt

MD5

- **Step 3**: Divide the 512 bits message blocks in to different words with 32 bits each $M_0, M_1, \ldots\ldots M_{15}$

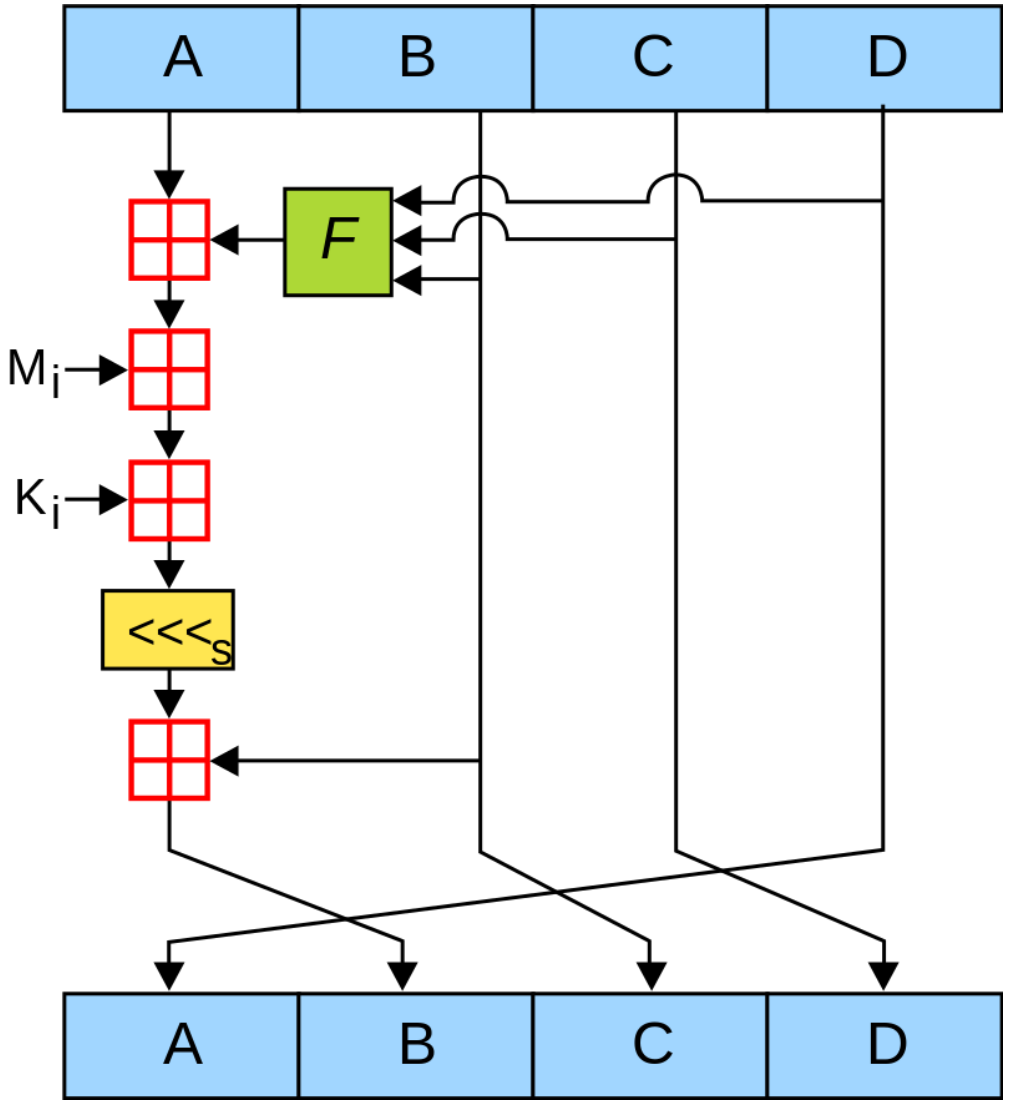
- **Step 4: Initialization:**
- A four-word buffer (A,B,C,D) is used to compute the message digest. Here each of A, B, C, D is a 32-bit register and each are initialized to specific values (128 bits)

- word A: 01 23 45 67
- word B: 89 ab cd ef
- word C: fe dc ba 98
- word D: 76 54 32 10
- Also denoted as $A_0B_0C_0D_0$



MD5

- **<u>Step 5: Processing:</u>**
- The padded message is divided into 512-bit blocks. Each block is subdivided into 16 words of 32-bit each (M_0 to M_{15}).
- Each block undergoes four groups of processing, each group consisting of 16 round operations. Total 64 Operations performed.
- <u>For round 0-15</u>
- F(B,C,D) = BC OR [NOT(B)D]
- <u>For round 16-31</u>
- F(B,C,D) = BD OR [C NOT(D)]



MD5

- For round 32-47
- F(B,C,D) = B XOR C XOR D
- For round 48-63
- F(B,C,D) = C XOR [B OR NOT (D)]
- **Step 6: Compute:**
- The final values of the registers A, B, C, and D are added with the initial values and concatenated to produce the 128-bit hash value.
- It is the message digest of MD5 algorithm for first block of message. This message digest is the input for next block. Perform similar operation for each block of message and final value of A, B, C, D is the result of MD5 algorithm.
- The final values of the registers A, B, C, and D are added with the initial values and concatenated to produce the 128-bit hash value. $A = A + A_0$, $B = B + B_0$, $C = C + C_0$ $D = D + D_0$

$$F(B,C,D) = (B \wedge C) \vee (\neg B \wedge D)$$
$$G(B,C,D) = (B \wedge D) \vee (C \wedge \neg D)$$
$$H(B,C,D) = B \oplus C \oplus D$$
$$I(B,C,D) = C \oplus (B \vee \neg D)$$

where $\wedge, \vee, \neg, \oplus$ are AND, OR, NOT, XOR

MD5

- In the first group of 16 rounds,

  $M_g = M_0, M_1, M_2 \ldots M_{15}$.
- In the second group of 16 rounds,

  $M_g = M_1, M_6, M_{11}, M_0, M_5, M_{10}, M_{15}, M_4, M_9, M_{14}, M_3, M_8, M_{13}, M_2, M_7, M_{12}$
- In the third group of 16 rounds,

  $M_g = M_5, M_8, M_{11}, M_{14}, M_1, M_4, M_7, M_{10}, M_{13}, M_0, M_3, M_6, M_9, M_{12}, M_{15}, M_2$
- In the fourth group of 16 rounds,

  $M_g = M_0, M_7, M_{14}, M_5, M_{12}, M_3, M_{10}, M_1, M_8, M_{15}, M_6, M_{13}, M_4, M_{11}, M_2, M_9$

- Formula (g = message word number, i = round number)

  Group 1: $g = i$.

  Group 2: $g = (5 \times i + 1) \bmod 16$.

  Group 3: $g = (3 \times i + 1) \bmod 16$.

  Group 4: $g = (7 \times i) \bmod 16$.

MD5

- Value of K_i for each steps are calculated using the formula: $abs(\sin(i + 1)) \times 232$

  and is given in the adjacent table:

K[0.. 3]	0xd76aa478, 0xe8c7b756, 0x242070db, 0xc1bdceee
K[4.. 7]	0xf57c0faf, 0x4787c62a, 0xa8304613, 0xfd469501
K[8..11]	0x698098d8, 0x8b44f7af, 0xffff5bb1, 0x895cd7be
K[12..15]	0x6b901122, 0xfd987193, 0xa679438e, 0x49b40821
K[16..19]	0xf61e2562, 0xc040b340, 0x265e5a51, 0xe9b6c7aa
K[20..23]	0xd62f105d, 0x02441453, 0xd8a1e681, 0xe7d3fbc8
K[24..27]	0x21e1cde6, 0xc33707d6, 0xf4d50d87, 0x455a14ed
K[28..31]	0xa9e3e905, 0xfcefa3f8, 0x676f02d9, 0x8d2a4c8a
K[32..35]	0xfffa3942, 0x8771f681, 0x6d9d6122, 0xfde5380c
K[36..39]	0xa4beea44, 0x4bdecfa9, 0xf6bb4b60, 0xbebfbc70
K[40..43]	0x289b7ec6, 0xeaa127fa, 0xd4ef3085, 0x04881d05
K[44..47]	0xd9d4d039, 0xe6db99e5, 0x1fa27cf8, 0xc4ac5665
K[48..51]	0xf4292244, 0x432aff97, 0xab9423a7, 0xfc93a039
K[52..55]	0x655b59c3, 0x8f0ccc92, 0xffeff47d, 0x85845dd1
K[56..59]	0x6fa87e4f, 0xfe2ce6e0, 0xa3014314, 0x4e0811a1
K[60..63]	0xf7537e82, 0xbd3af235, 0x2ad7d2bb, 0xeb86d391

0x denotes HEX representation.

- The left rotation amount for each steps is presented below:

```
s[ 0..15] = {7,12,17,22,7,12,17,22,7,12,17,22,7,12,17,22}
s[16..31] = {5,9,14,20,5,9,14,20,5,9,14,20,5,9,14,20}
s[32..47] = {4,11,16,23,4,11,16,23,4,11,16,23,4,11,16,23 }
s[48..63] = {6,10,15,21,6,10,15,21,6,10,15,21,6,10,15,21 }
```

https://cs.indstate.edu/~fsagar/doc/paper.pdf

MD5

- MD5 Round 1

/* [abcd k s i] denotes
a = b + ((a + F(b,c,d) + X[k] + T[i]) <<< s). */
/* 16 operations. */
[ABCD 0 7 1] [DABC 1 12 2] [CDAB 2 17 3] [BCDA 3 22 4]
[ABCD 4 7 5] [DABC 5 12 6] [CDAB 6 17 7] [BCDA 7 22 8]
[ABCD 8 7 9] [DABC 9 12 10] [CDAB 10 17 11] [BCDA 11 22 12]
[ABCD 12 7 13] [DABC 13 12 14] [CDAB 14 17 15] [BCDA 15 22 16]

Constant added, varies

MD5

- MD5 Round 2

/* [abcd k s i] denotes
a = b + ((a + G(b,c,d) + X[k] + T[i]) <<< s). */
/*16 operations. */
[ABCD 1 5 17] [DABC 6 9 18] [CDAB 11 14 19] [BCDA 0 20 20]
[ABCD 5 5 21] [DABC 10 9 22] [CDAB 15 14 23] [BCDA 4 20 24]
[ABCD 9 5 25] [DABC 14 9 26] [CDAB 3 14 27] [BCDA 8 20 28]
[ABCD 13 5 29] [DABC 2 9 30] [CDAB 7 14 31] [BCDA 12 20 32]

not reusing shift amounts across rounds

MD5

- MD5 Round 3

```
/* Let [abcd k s i] denotes
a = b + ((a + H(b,c,d) + X[k] + T[i]) <<< s). */
/* Do the following 16 operations. */
[ABCD 5 4 33] [DABC 8 11 34] [CDAB 11 16 35] [BCDA 14 23 36]
[ABCD 1 4 37] [DABC 4 11 38] [CDAB 7 16 39] [BCDA 10 23 40]
[ABCD 13 4 41] [DABC 0 11 42] [CDAB 3 16 43] [BCDA 6 23 44]
[ABCD 9 4 45] [DABC 12 11 46] [CDAB 15 16 47] [BCDA 2 23 48]
```

Word ordering altered from round 2 to greater extent than in MD4

MD5

- MD5 Round 4

```
/* [abcd k s i] denotes
   a = b + ((a + I(b,c,d) + X[k] + T[i]) <<< s). */
/* 16 operations. */
[ABCD 0 6 49] [DABC 7 10 50] [CDAB 14 15 51]
[BCDA 5 21 52] [ABCD 12 6 53] [DABC 3 10 54]
[CDAB 10 15 55] [BCDA 1 21 56] [ABCD 8 6 57]
[DABC 15 10 58] [CDAB 6 15 59] [BCDA 13 21 60]
[ABCD 4 6 61] [DABC 11 10 62] [CDAB 2 15 63] [BCDA 9 21 64]
```

One more round than MD4

Secure Hash Algorithms (SHA)

- In recent years, the most widely used hash function has been the Secure Hash Algorithm (SHA).
- SHA was developed by the National Institute of Standards and Technology (NIST) and published as a federal information processing standard (FIPS 180) in 1993.
- When weaknesses were discovered in SHA, a revised version was issued as FIPS 180-1 in 1995 and is generally referred to as SHA-1.
- SHA-1 produces a hash value of 160 bits.

Secure Hash Algorithms (SHA)

- In 2002, NIST produced a revised version of the standard, FIPS 180–2, that defined three new versions of SHA, with hash value lengths of 256, 384, and 512 bits, known as SHA-256, SHA-384, and SHA-512.
- These new versions, collectively known as SHA-2, have the same underlying structure and use the same types of modular arithmetic and logical binary operations as SHA-1.
- SHA-2, particularly the 512-bit version, would appear to provide unassailable security.
- However, because of the structural similarity of SHA-2 to SHA-1, NIST decided to standardize a new hash function that very different from SHA-2 and SHA-1. This new hash function, known as SHA-3, was published in 2012.

Secure Hash Algorithms (SHA)

- In 2005, NIST announced the intention to phase out approval of SHA-1 and move to a reliance on the other SHA versions by 2010. Shortly thereafter, a research team described an attack in which two separate messages could be found that deliver the same SHA-1 hash using 269 operations, far fewer than the 280 operations previously thought needed to find a collision with an SHA-1 hash. This result has hastened (speed up) the transition to the other versions of SHA.
- SHA-1 is now considered insecure since 2005. Major tech giants' browsers like Microsoft, Google, Apple and Mozilla have stopped accepting SHA-1 SSL certificates by 2017.

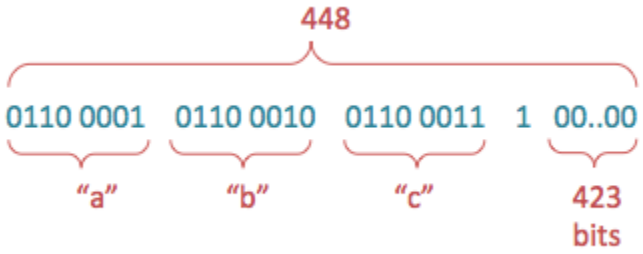
Table 11.3 Comparison of SHA Parameters

	SHA-1	SHA-224	SHA-256	SHA-384	SHA-512
Message Digest Size	160	224	256	384	512
Message Size	$< 2^{64}$	$< 2^{64}$	$< 2^{64}$	$< 2^{128}$	$< 2^{128}$
Block Size	512	512	512	1024	1024
Word Size	32	32	32	64	64
Number of Steps	80	64	64	80	80

Note: All sizes are measured in bits.

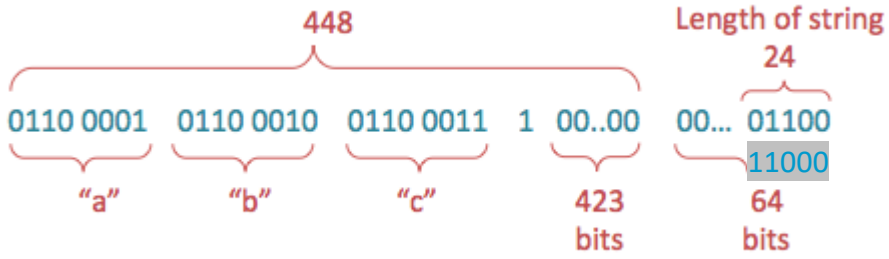
SHA-1

- Hash Value: 160 bits
- Message: Variable Length: multiple of 512 bits
- Sub Block size/Word=32bit
- 512/32=16 total Sub blocks/Words
- No. Of Rounds=4
- **<u>Step 1: Append Padding Bits:</u>**
- The Original Message is "padded" with a 1 and as many 0's as necessary at the end to bring the message length to 64 bits fewer than an even multiple of 512.
- Example: Suppose the message 'abc' were to be encoded using SHA-1. The message can be converted to binary using ASCII Table as :
- In HEX: 61 62 63
- In Binary: 01100001 01100010 01100011



SHA-1

- **<u>Step 2: Append Length:</u>**
- 64 bits are appended to the end of the padded message to define the length of the original message (before padding in step 1). These bits hold the binary format of 64 bits indicating the length of the original message.
- The complete message's length after Step 2 is a multiple of 512.
- For our example:



- **<u>Step 3: *Divide the input into 512-bit blocks*</u>**

SHA-1

- **<u>Step 4: Initialize Hash Variables:</u>**
- Make five hash variables A, B, C, D, E with 32 bits each, total 160 bits which is the length of finale hash code. The combination of hash variable A-E is considered as a single register.
- These five registers are initialized to the following 32-bit integers (in hexadecimal notation).

A	B	C	D	E

$A = H_0 = 67\ 45\ 23\ 01$

$B = H_1 = ef\ cd\ ab\ 89$

$C = H_2 = 98\ ba\ dc\ fe$

$D = H_3 = 10\ 32\ 54\ 76$

$E = H_4 = c3\ d2\ e1\ f0$

https://www.nic.ad.jp/ja/tech/ipa/RFC3174EN.html

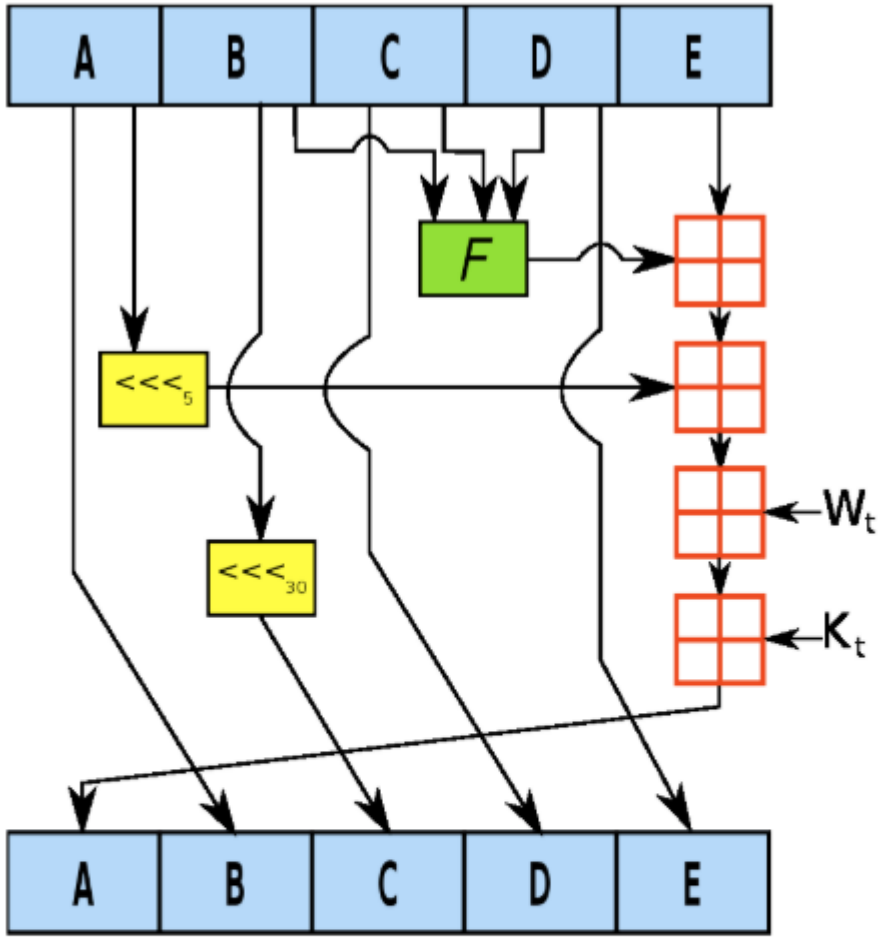
SHA-1

- **<u>Step 5: Process the Message</u>**
- The heart of the algorithm is a module that consists of 80 rounds.
- SHA-1 requires 80 processing functions defined as:

f(t;B,C,D) = (B AND C) OR ((NOT B) AND D)	(0 <= t <= 19)
f(t;B,C,D) = B XOR C XOR D	(20 <= t <= 39)
f(t;B,C,D) = (B AND C) OR (B AND D) OR (C AND D)	(40 <= t <=59)
f(t;B,C,D) = B XOR C XOR D	(60 <= t <= 79)

- SHA-1 requires 80 processing constant words defined as:

K_t: K(t) = 0x5A827999	(0 <= t <= 19)
K_t: K(t) = 0x6ED9EBA1	(20 <= t <= 39)
K_t: K(t) = 0x8F1BBCDC	(40 <= t <= 59)
K_t: K(t) = 0xCA62C1D6	(60 <= t <= 79)



One iteration within the SHA-1 compression function:
A, B, C, D and E are 32-bit words of the state;
F is a nonlinear function that varies;
$\lll_n$ denotes a left bit rotation by *n* places;
n varies for each operation;
W_t is the expanded message word of round t;
K_t is the round constant of round t;
$\boxplus$ denotes addition modulo 2^{32}.

SHA-1

- Take a message block (512 bits) and input to Message Scheduling Algorithm (MSA) which converts the 512 bits size message blocks in to 16 words (w_0 leftmost – w_{15} rightmost). Each word contains 32 bits of message.
- For t = 16 to 79
- $w_t = w_{t-3}$ XOR w_{t-8} XOR w_{t-14} XOR w_{t-16}.
- Our task here is to calculate A, B, C, D and E for 80 rounds. After each rounds:

  A ← 5 bits left rotation of A + f(t;B,C,D) + E + w_t + K_t

  B ← A

  C ← 30 bits left rotation of B

  D ← C

  E ← D

Where,

$W_t = W_{n-3} \oplus W_{n-8} \oplus W_{n-14} \oplus W_{n-16}$ is the tth 32 bits word block

W(t) is the expanded message word of round t;
K(t) is the round constant of round t;

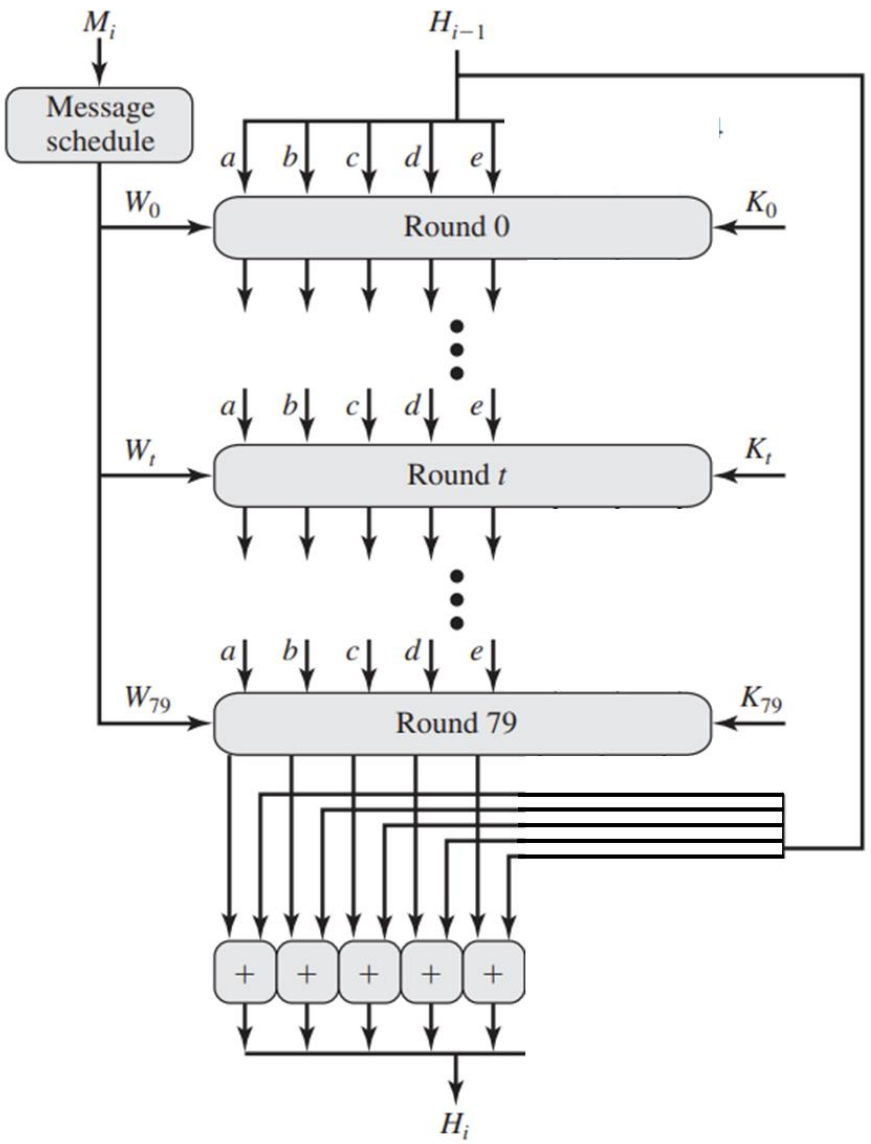
In each iteration, these values are updated as

$$
\begin{aligned}
B &= old\ A \\
C &= old\ B \lrcorner 30 \\
D &= old\ C \\
E &= old\ D \\
A &= E + (A \lrcorner 5) + W_t + K_t + f(t, B, C, D)
\end{aligned}
$$

SHA-1

- The output of the eightieth round is added to the input to the first round to produce the hash output of the round .The addition is done independently for each of the five words.

- **<u>Step 6: Output:</u>**
- After all blocks (each of 512 bits) have been processed, the hash output of the final block is the 160-bit **message digest**.



Creation of 80-word input W_t

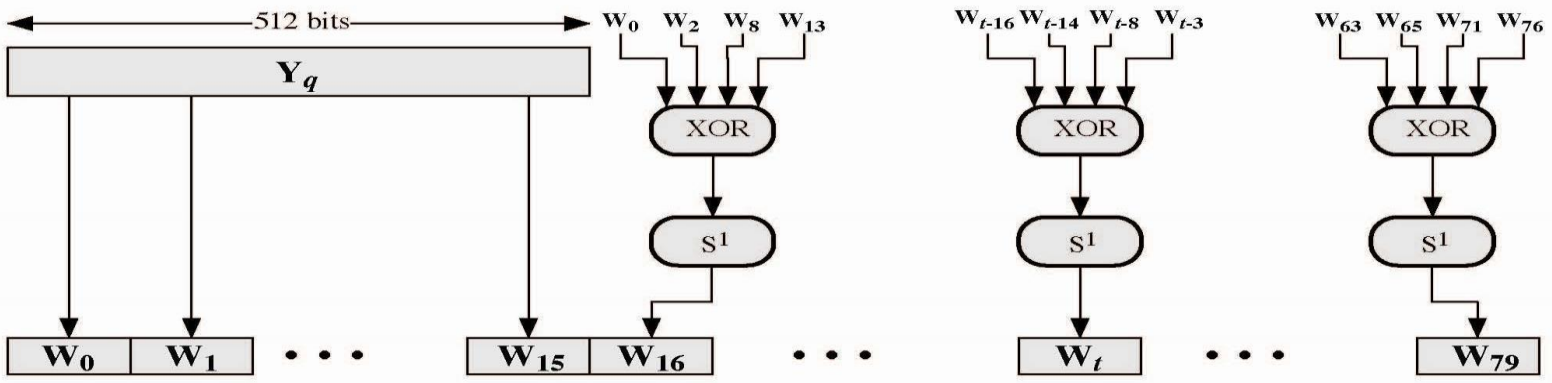
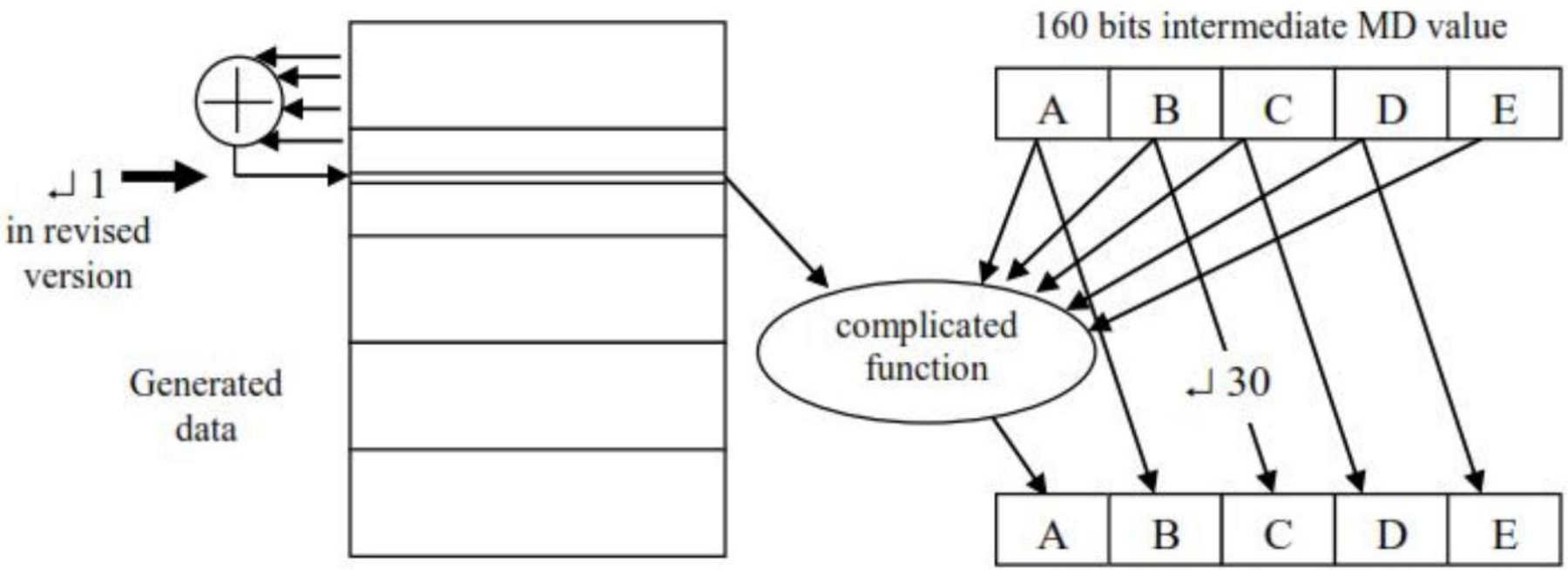


Figure 12.7 Creation of 80-word Input Sequence for SHA-1 Processing of Single Block

SHA1 Round Operation




```
A <- (E+f(t,B,C,D)+(A<<5)+Wt+Kt)
B <- A
C <- (B<<30) circular left shift
D <- C
E <- D
```

SHA-1 Compression/Complicated Function

$$A \leftarrow (E+f(t,B,C,D)+(A<<5)+W_t+K_t)$$

$K_t = \mathbf{floor}(2^{30}\sqrt{2}) = 5a827999_{16}$ if $0 \le t \le 19$.

$K_t = \mathbf{floor}(2^{30}\sqrt{3}) = 6ed9eba1_{16}$ if $20 \le t \le 39$.

$K_t = \mathbf{floor}(2^{30}\sqrt{5}) = 8f1bbcdc_{16}$ if $40 \le t \le 59$

$K_t = \mathbf{floor}(2^{30}\sqrt{10}) = ca62c1d6_{16}$ if $60 \le t \le 79$.

Process F(t) in each SHA-1 round

- where f can be expressed as:

ROUND 1: (BC) v (~B)D	**(t= 0 to 19)**
ROUND 2: B ⊕ C ⊕ D	**(t= 20 to 39)**
ROUND 3: (BC) v (BD) v (CD)	**(t= 40 to 59)**
ROUND 4: B ⊕ C ⊕ D	**(t= 60 to 79)**

SHA-2 (SECURE HASH ALGORITHM 2)

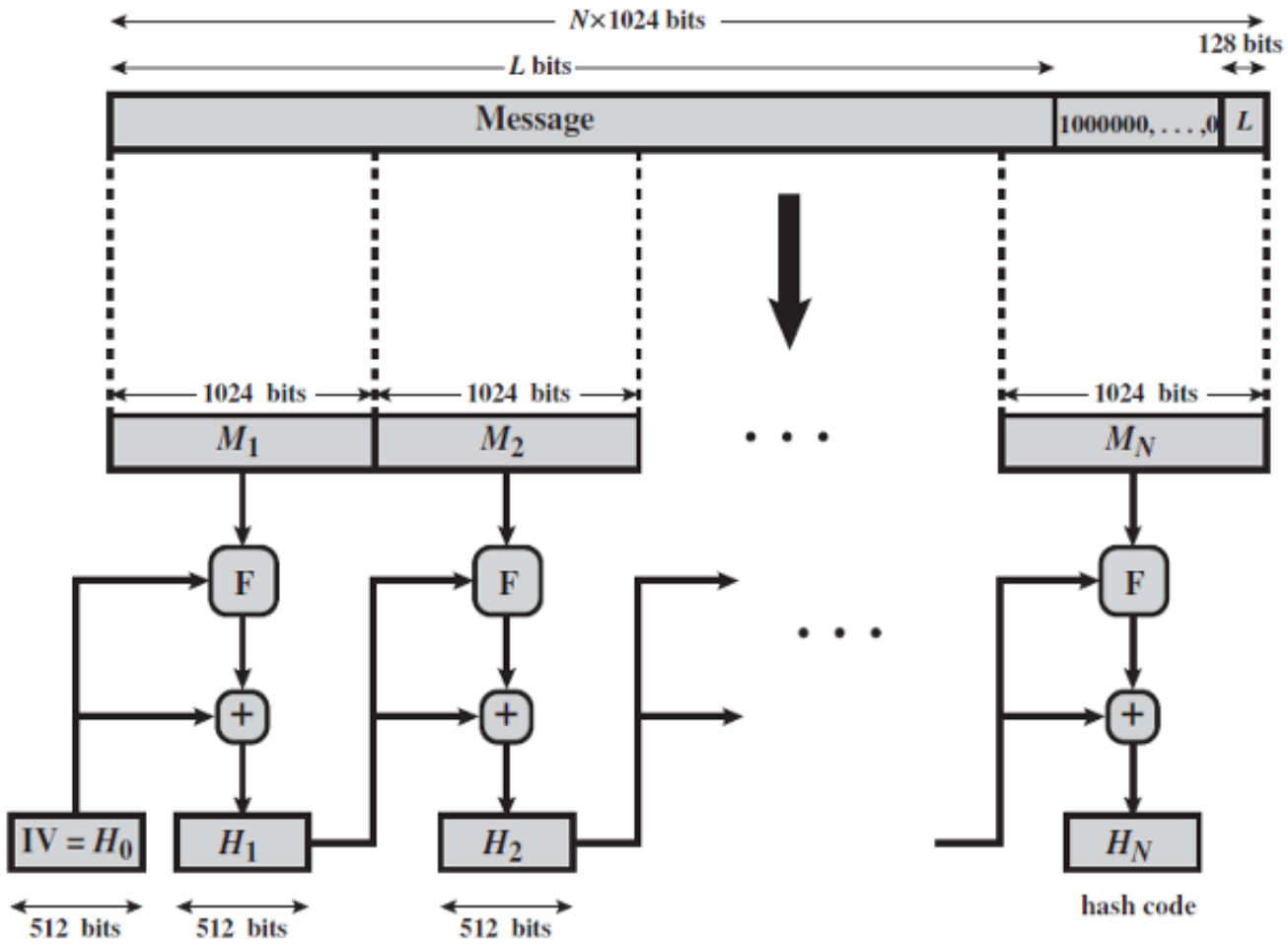
- SHA-1 produces a hash value of 160 bits. In 2002, NIST produced a revised version of the standard, FIPS 180-2, that defined three new versions of SHA, with hash value lengths of 256, 384, and 512 bits, known as SHA-256, SHA-384, and SHA-512, respectively. Collectively, these hash algorithms are known as SHA-2.
- SHA-224 – This version of SHA-2 produces a 224-bit hash. It has a block size of 512 bits, and the initial input is divided into 32-bit words for processing. The initialization variables are also 32 bits in length, as are the constants, K. Each block of data goes through 64 rounds of operations before the final hash (or the intermediate hash, in cases where multiple blocks of data are being processed) is produced.
- SHA-256 – SHA-256 results in a 256-bit hash and has a 512-bit block size. The message input is processed in 32-bit words, while the initialization variables and constants are also 32 bits in length. SHA-256 also involves 64 rounds.
- SHA-384 – This version produces a 384-bit hash. It differs from the prior two in that it has a 1,024-bit block size. It also varies in that it has 64-bit words, initialization variables and constants. Instead of 64 rounds, it requires 80 rounds of processing for each block of message data.

SHA-2 (SECURE HASH ALGORITHM 2)

- SHA-512 – SHA-512 results in a 512-bit hash. Apart from that, it's much like SHA-384 in that it has a 1,024-bit block size, 64-bit words, 64-bit initialization variables and 64-bit constants. However, the particular initialization variables it begins with are different from those in SHA-384. It also involves 80 rounds.
- SHA-512/224 – This version is much like SHA-512, except that it results in a truncated hash of 224-bits. This means that it involves a process that is largely the same, except that only the left-most 224 bits are taken as the hash, while the rest is discarded. The block size is also 1024 bits, while the words, constants and initialization variables are all 64-bits long. However, the initialization variables are different from those used in SHA-512 or SHA-384. SHA-512/224 also requires 80 rounds for each block of message data.
- SHA-512/256 – Like SHA-512/224, this iteration is also similar to SHA-512, except it produces a truncated 256-bit hash by only taking the left-most 64 bits. It has a 1,024 bit block size, as well as 64-bit words, constants and initialization variables. SHA-512/256 also has its own set of initialization variables. It involves 80 rounds.
- SHA 512 is studied in detail.

SHA-512

SHA-512 The algorithm takes as input a message with a maximum length of less than 2^128 bits and produces as output a 512-bit message digest. The input is processed in 1024-bit block. Following Figure depicts the overall processing of a message to produce a digest.



The processing consists of the following steps.

Step 1- Append padding bits

- The message is padded so that its length is congruent to 896 modulo 1024 [length $\equiv$ 896(mod 1024)].
- Padding is always added, even if the message is already of the desired length. Thus, the number of padding bits is in the range of 1 to 1024.
- The padding consists of a single 1 bit followed by the necessary number of 0 bits.

Step 2 - Append length

- A block of 128 bits is appended to the message. This block is treated as an unsigned 128-bit integer (most significant byte first) and contains the length of the original message (before the padding).

- The outcome of the first two steps yields a message that is an integer multiple of 1024 bits in length. In Figure 11.9, the expanded message is represented as the sequence of 1024-bit blocks M_1, M_2, ... , M_N, so that the total length of the expanded message is N * 1024 bits.

Message block and digest word

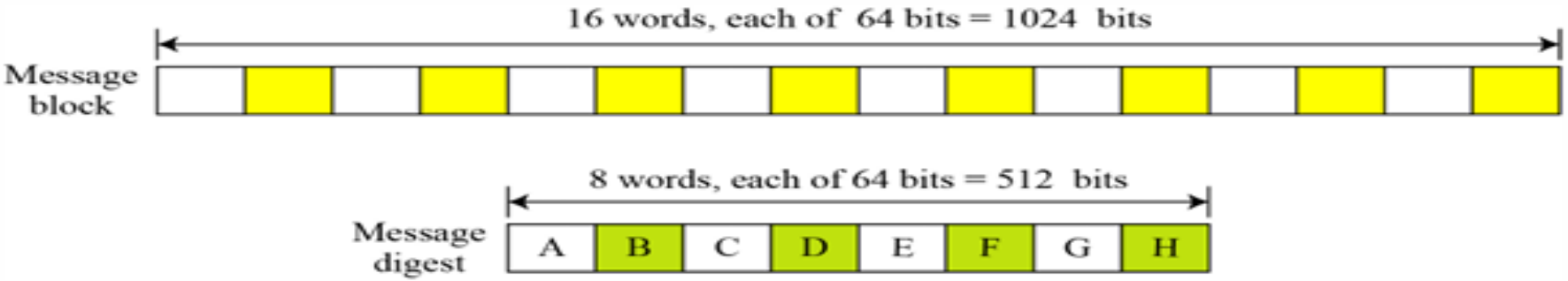


Figure: A message block and the digest as words

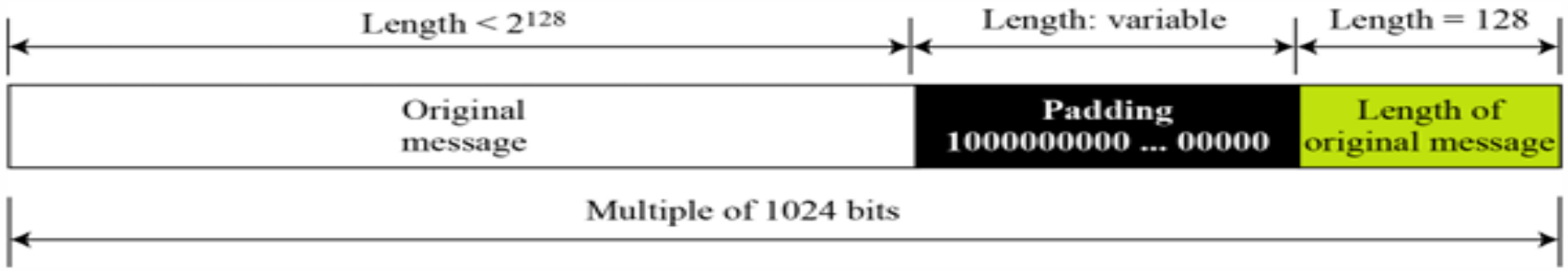


Figure: Padding and length field

Step 3 - Initialize hash buffer

- A 512-bit buffer is used to hold intermediate and final results of the hash function. The buffer can be represented as eight 64-bit registers (a, b, c, d, e, f, g, h).
- These registers are initialized to the following 64-bit integers (hexadecimal values):

```
a = 6A09E667F3BCC908      e = 510E527FADE682D1

b = BB67AE8584CAA73B      f = 9B05688C2B3E6C1F

c = 3C6EF372FE94F82B      g = 1F83D9ABFB41BD6B

d = A54FF53A5F1D36F1      h = 5BE0CD19137E2179
```

- These values are stored in ***big-endian*** format, which is the most significant byte of a word in the low-address (leftmost) byte position.
- These words were obtained by taking the *first sixty-four bits of the fractional parts of the square roots of the first eight prime numbers.*

Step 4 - Process message

- Process message In 1024-bit (128-word) blocks.
- The heart of the algorithm is a module that consists of 80 rounds; this module is labeled F in figure above.
- The logic is illustrated in following Figure

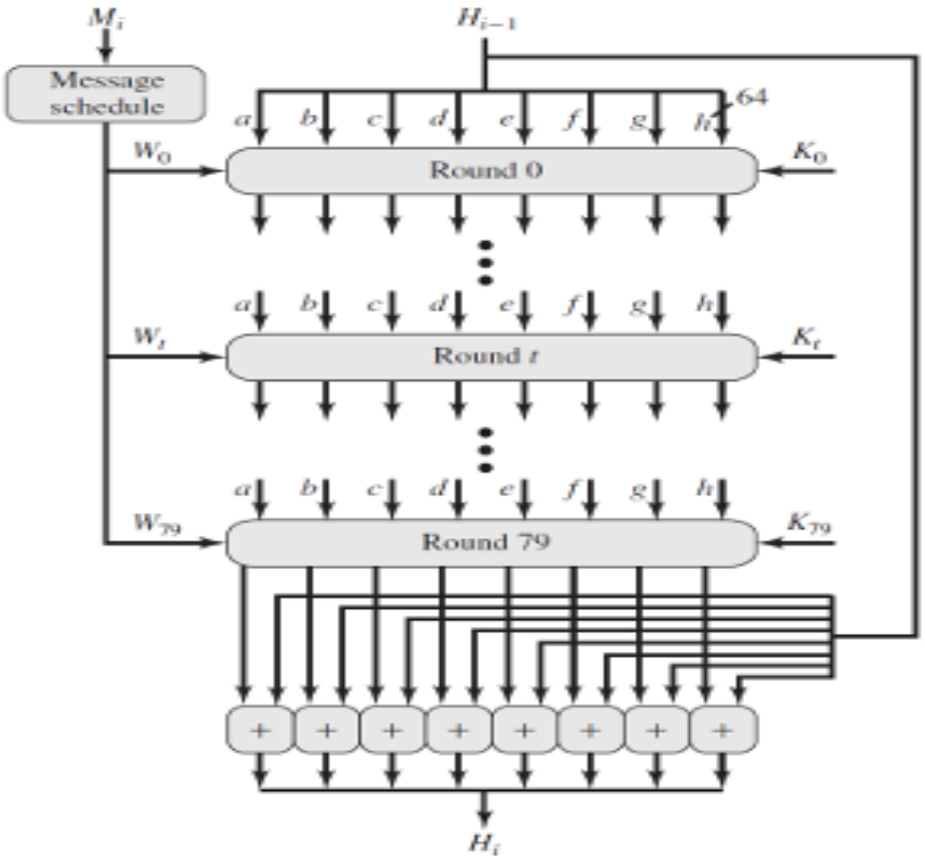


Figure: SHA-512 Processing of a Single 1024-Bit Block.

- Each round takes as input the 512-bit buffer value, abcdefgh, and updates the contents of the buffer.
- At input to the first round, the buffer has the value of the intermediate hash value, H_{i-1}.
- Each round t makes use of a 64-bit value W_t, derived from the current 1024-bit block being processed (M_i). These values are derived using a message schedule.
- Each round also makes use of an additive constant K_t, where $0 \leq t \leq 79$ indicates one of the 80 rounds.These words represent the *first 64 bits of the fractional parts of the cube roots of the first 80 prime numbers.* The constants provide a "randomized" set of 64-bit patterns, which should eliminate any regularities in the input data.
- The output of the eightieth round is added to the input to the first round (H_{i-1}) to produce H_i. The addition is done independently for each of the eight words in the buffer with each of the corresponding words in H_{i-1}, using addition modulo 2^{64}

Step 5 – Output

After all N 1024-bit blocks have been processed, the output from the Nth stage is the 512-bit message digest.

We can summarize the behavior of SHA-512 as follows:

$H_0 = IV$

$H_i = SUM_{64}(H_{i-1}, abcdefgh_i)$

$MD = H_N$

where

IV = initial value of the abcdefgh buffer, defined in step 3

$abcdefgh_i$ = the output of the last round of processing of the ith message block

N = the number of blocks in the message (including padding and length fields)

SUM_{64} = addition modulo 2^{64} performed separately on each word of the pair of inputs

MD = final message digest value

$$T_1 = h + \mathrm{Ch}(e, f, g) + \left(\sum_1^{512} e\right) + W_t + K_t$$

$$T_2 = \left(\sum_0^{512} a\right) + \mathrm{Maj}(a, b, c)$$

$$h = g$$

$$g = f$$

$$f = e$$

$$e = d + T_1$$

$$d = c$$

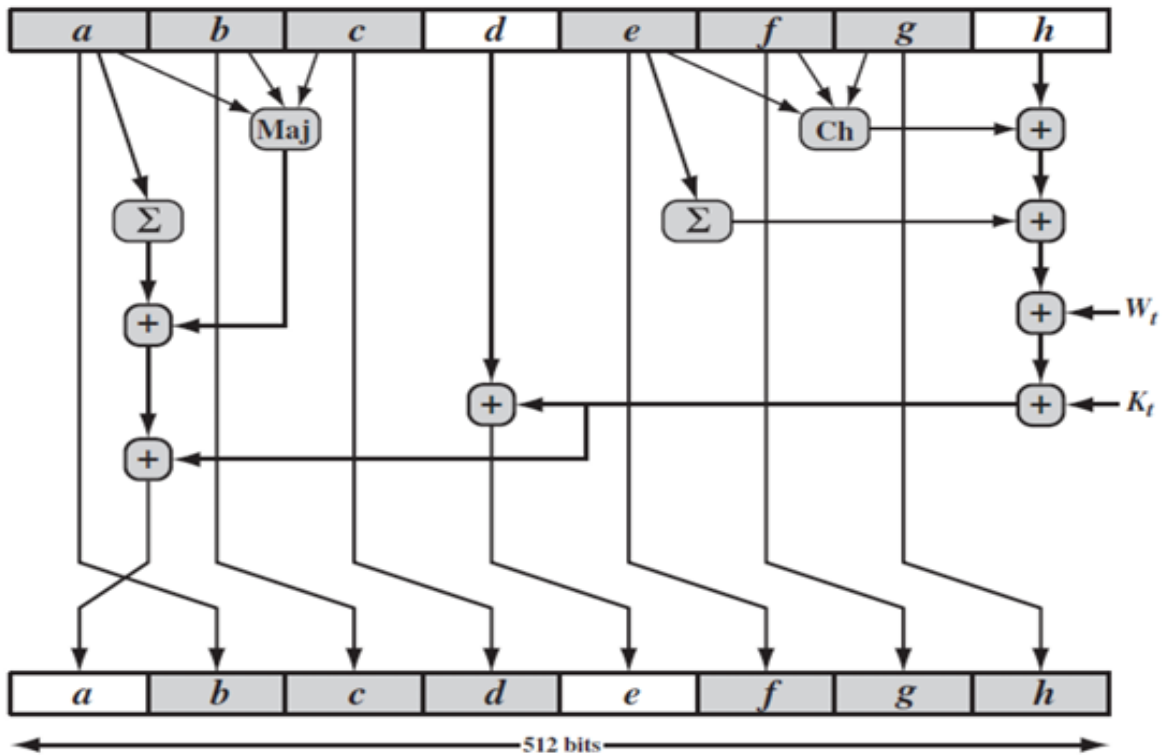
$$c = b$$

$$b = a$$

$$a = T_1 + T_2$$

SHA-512 Round Function

Let us look in more detail at the logic in each of the 80 steps of the processing of one 512-bit block in following figure



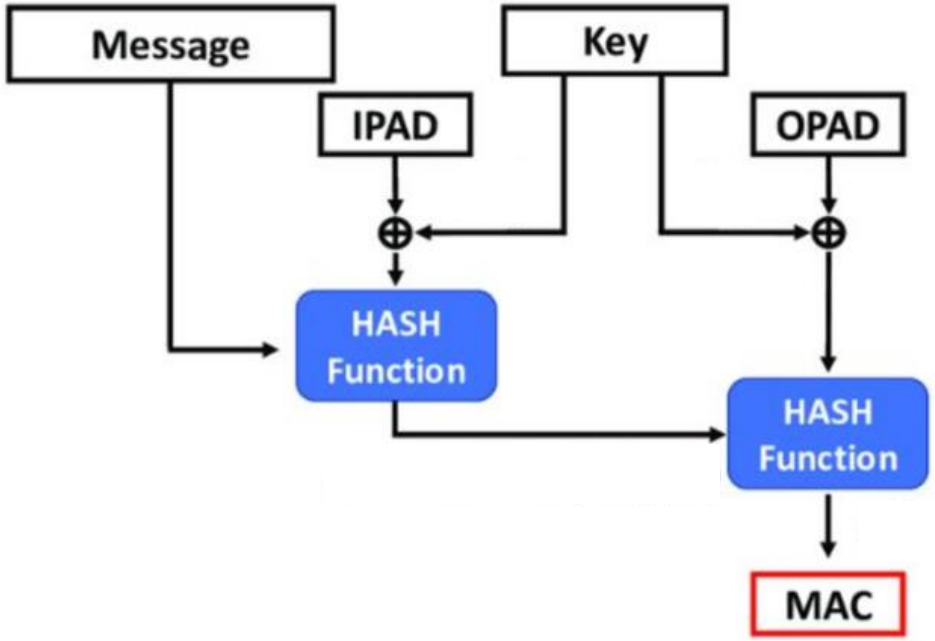
Where,

$$
\begin{aligned}
t &= \text{step number; } 0 \le t \le 79 \\
\mathrm{Ch}(e, f, g) &= (e \text{ AND } f) \oplus (\text{NOT } e \text{ AND } g) \\
&\quad \textit{the conditional function: If } e \textit{ then } f \textit{ else } g \\
\mathrm{Maj}(a, b, c) &= (a \text{ AND } b) \oplus (a \text{ AND } c) \oplus (b \text{ AND } c) \\
&\quad \textit{the function is true only of the majority (two or three) of the} \\
&\quad \textit{arguments are true} \\
\left(\sum_0^{512} a\right) &= \mathrm{ROTR}^{28}(a) \oplus \mathrm{ROTR}^{34}(a) \oplus \mathrm{ROTR}^{39}(a) \\
\left(\sum_1^{512} e\right) &= \mathrm{ROTR}^{14}(e) \oplus \mathrm{ROTR}^{18}(e) \oplus \mathrm{ROTR}^{41}(e) \\
\mathrm{ROTR}^n(x) &= \text{circular right shift (rotation) of the 64-bit argument } x \text{ by } n \text{ bits}
\end{aligned}
$$

$$
\begin{aligned}
W_t &= \text{a 64-bit word derived from the current 1024-bit input block} \\
K_t &= \text{a 64-bit additive constant} \\
+ &= \text{addition modulo } 2^{64}
\end{aligned}
$$

HMAC (Hash function-based MAC)

- In recent years, there has been increased interest in developing a MAC derived from a cryptographic hash function.
- The cryptographic authentication method which uses a hash function, and a secret key is known as a Hash-based Message Authentication Code (or HMAC).



- Input to HMAC:

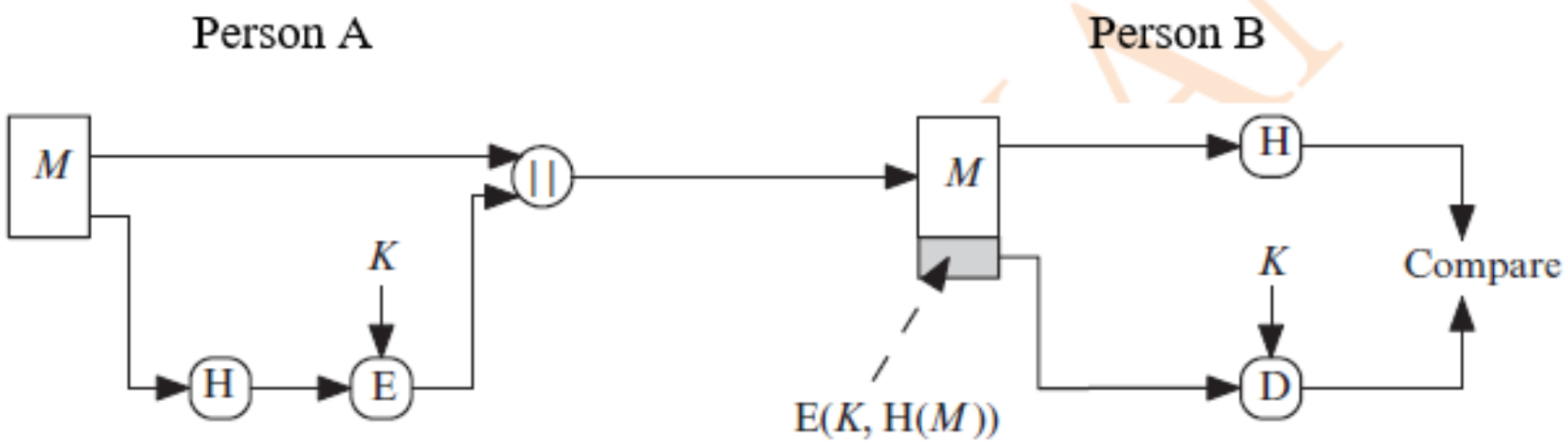
  A message M and a secret key K.
- Output:

  A hash value h that acts as the MAC.

HMAC (Hash function-based MAC)

The cryptographic authentication method which uses a hash function and a secret key is known as a Hash-based Message Authentication Code (or HMAC). You can utilize shared secrets and HMAC to accomplish authentication and confirm that the data is accurate and valid (both sender and recipient use the same key to generate and check the MAC).

Consider an information exchange between two persons, Person A(sender) and Person B (recipient), Person A sends a shared secret with Person B and Person B tries to authenticate.



HMAC (Hash function-based MAC)

1. First of all perform hashing operation to message using any SHA algorithm.
2. Generated Hash Code from hashing is Encrypted using symmetric key encryption and generates a cipher text.
3. Append cipher text to original message and send to receiver.
4. Receiver Decrypt the cipher text using symmetric secret key and get a Hash Code H1
5. Perform same hash operation as sender to the received message and generate a Hash Code H2
6. If both the Hash Code H1 and H2 are equal then message is accept otherwise message is rejected.

HMAC

H = embedded hash function (e.g., MD5, SHA-1, RIPEMD-160)

IV = initial value input to hash function

M = message input to HMAC (including the padding specified in the embedded hash function)

Y_i = i th block of M, $0 \le i \le (L-1)$

L = number of blocks in M

b = number of bits in a block

n = length of hash code produced by embedded hash function

K = secret key; recommended length is $\ge n$; if key length is greater than b, the key is input to the hash function to produce an n-bit key

K^+ = K padded with zeros on the left so that the result is b bits in length

ipad = 00110110 (36 in hexadecimal) repeated $b/8$ times

opad = 01011100 (5C in hexadecimal) repeated $b/8$ times

Then HMAC can be expressed as

$$\text{HMAC}(K, M) = \text{H}[(K^+ \oplus \text{opad}) \,||\, \text{H}[(K^+ \oplus \text{ipad}) \,||\, M]]$$

where || denotes concatenation.

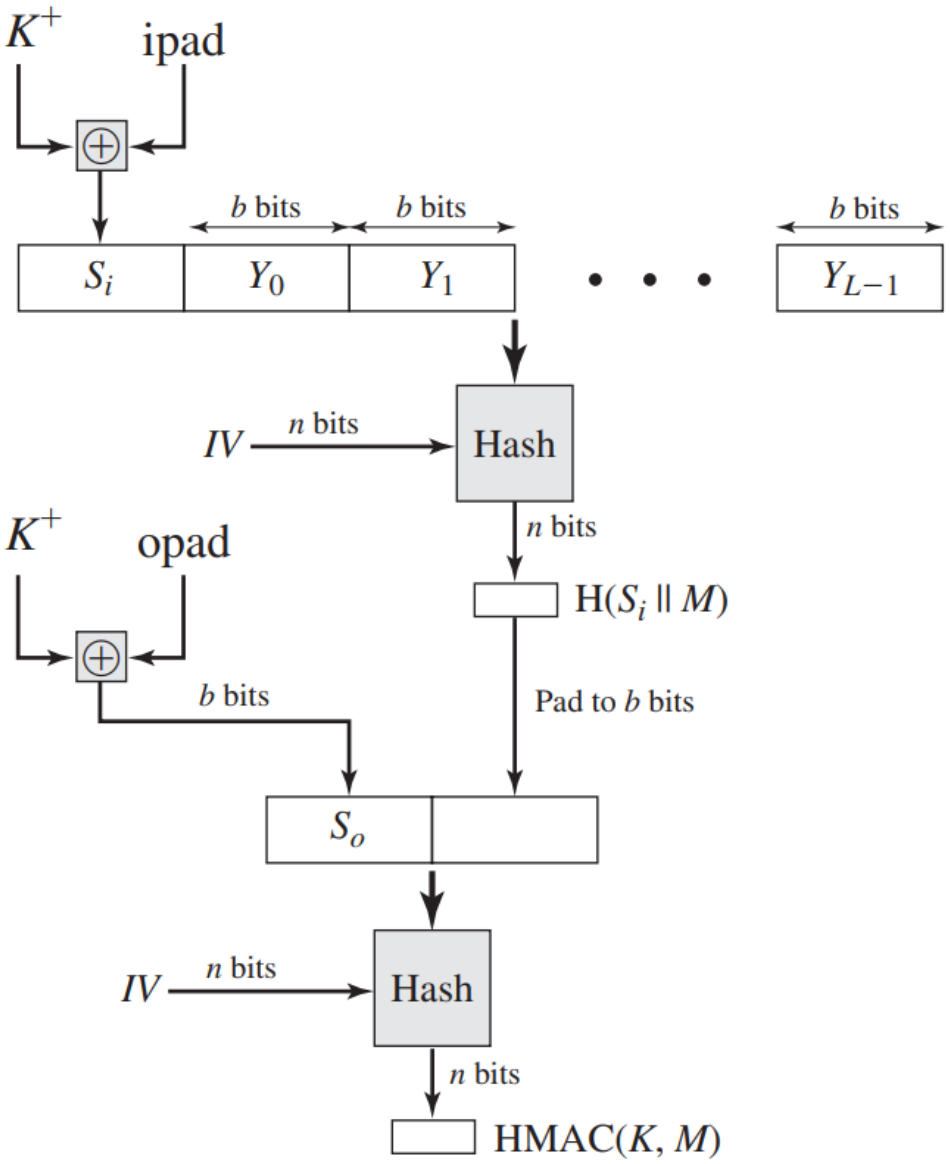


Figure 12.5 HMAC Structure

HMAC

- ***M*** = Input Message.
- Y_i = Message Block of length ***b***. There are ***L*** blocks.
- ***n*** = Length of Hash Code.
- ***K*** = Key.
- K^+ = Key of length ***b***. If given key ***K*** is larger than ***b***-bits, it is passed through the Hash function to generate key of ***n***-bits. Then, zeros are appended to the left end of ***K*** or hashed ***K*** to create K^+ which is a b-bit string.
- S_i = Inner secret key. It is obtained by bitwise-XOR of K^+ and inner padding value(***ipad***) (all of length ***b***-bits).
- Append ***M*** to S_i.
- Apply Hash function to the appended stream.
- S_o = Outer secret key. It is obtained by bitwise-XOR of K^+ and outer padding value(***opad***) (all of length ***b***-bits).
- Append the result from Hash function to S_o.
- Apply Hash function to the stream generated.
- Output will be the result (***HMAC***).

$$\mathrm{HMAC}(K, M) = \mathrm{H}[(K^+ \oplus \mathrm{opad}) \,||\, \mathrm{H}[(K^+ \oplus \mathrm{ipad}) \,||\, M]]$$

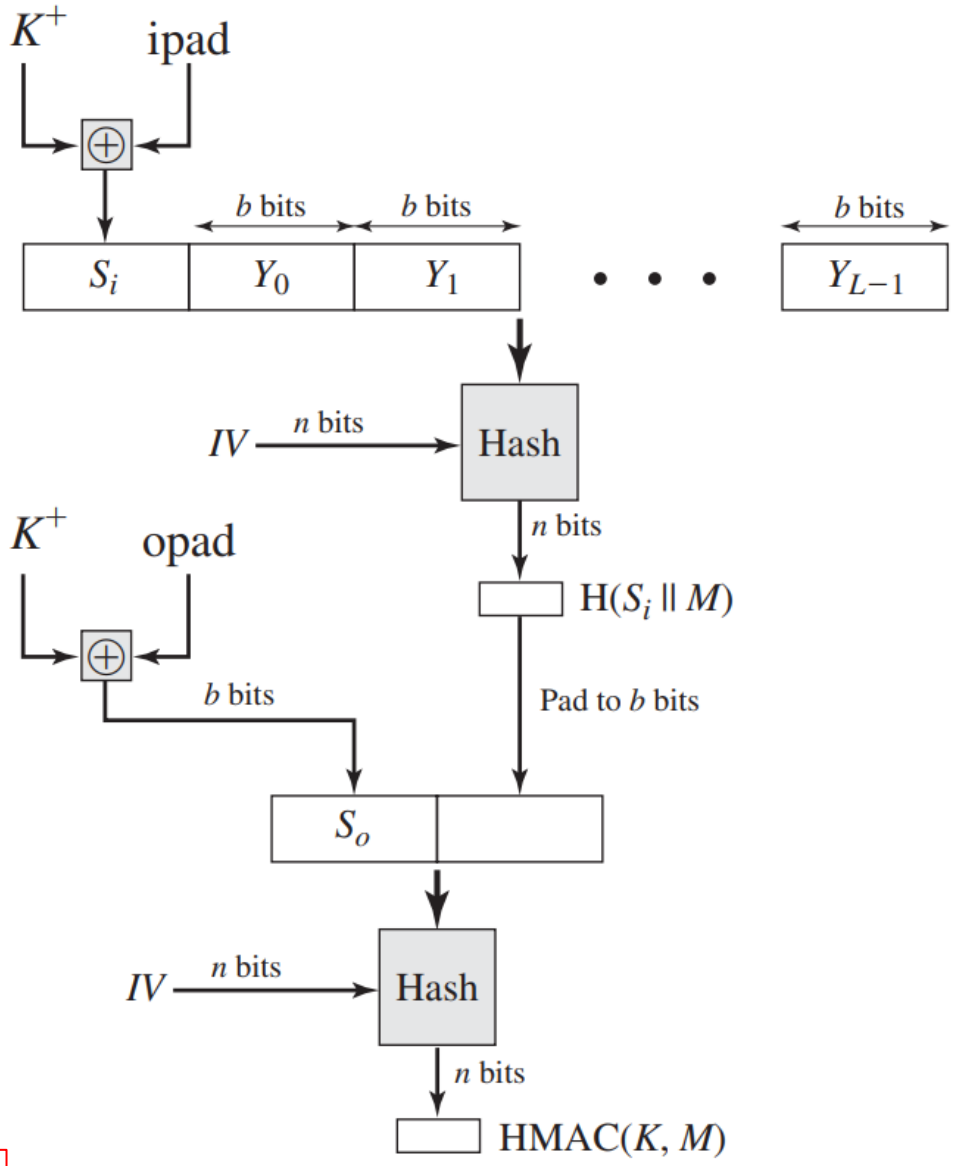


Figure 12.5 HMAC Structure

HMAC

- We can describe the algorithm as follows.

1. Append zeros to the left end of K to create a b-bit string K^+ .
2. XOR (bitwise exclusive-OR) with ipad to produce the b-bit block S_i.
3. Append M to S_i.
4. Apply H to the stream generated in step 3.
5. XOR K^+ with opad to produce the b-bit block S_0.
6. Append the hash result from step 4 to S_0.
7. Apply H to the stream generated in step 6 and output the result.

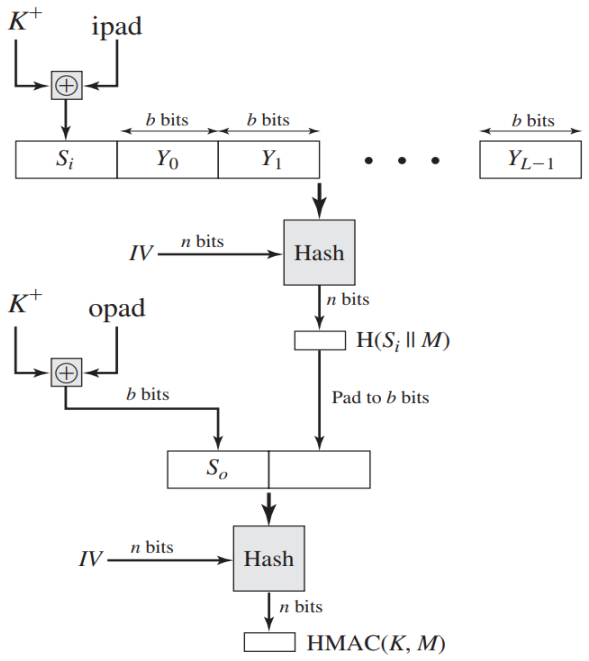


Figure 12.5 HMAC Structure

$S_i = K^+ \oplus ipad$ where K^+ is nothing but K padded with zeros on the left so that the result is b bits in length

$S_o = K^+ \oplus opad$ where ipad and opad are 00110110 and 01011100 respectively taken b/8 times repeatedly.

$MD' = H(S_i || M)$

$MD = H(S_o || MD')$ or $MD = H(S_o || H(S_i || M))$

HMAC

- The steps involved in HIMAC (Hash-based Message Authentication Code) are as follows:
- Key Padding: If the key size exceeds the block size of the hash function being used (e.g., SHA-256), the key is hashed to produce a fixed-size key.
- Padding: The message is padded to ensure its length is a multiple of the block size of the hash function.
- Inner Padding: The key is XORed with a specific value (often referred to as the inner padding).
- Hashing: The result of step 3 is concatenated with the padded message, and the combined result is hashed using the chosen hash function.
- Outer Padding: The output from step 4 is XORed with another specific value (outer padding).
- Second Hashing: The result of step 5 is concatenated with the hash produced in step 4 and hashed again using the chosen hash function.
- Output: The final hash output serves as the HMAC.

Digital Signature

- A digital signature is an authentication mechanism that enables the creator of a message to attach a code that acts as a signature.
- Typically the signature is formed by taking the hash of the message and encrypting the message with the creator's private key.
- The signature guarantees the source and integrity of the message.
- The digital signature standard (DSS) is an NIST standard that uses the secure hash algorithm (SHA).
- The most important development from the work on public-key cryptography is the digital signature.
- The digital signature provides a set of security capabilities that would be difficult to implement in any other way.

Digital Signature

Digital signature is an electronic signature that can be used to authenticate the identity of the sender of a message and to ensure that the original content of the message or document that has been sent is unchanged.

Digital signature schemes normally gives two algorithms, one for signing which involves the user's secret or private key and one for verifying signatures which involves the user's public key.

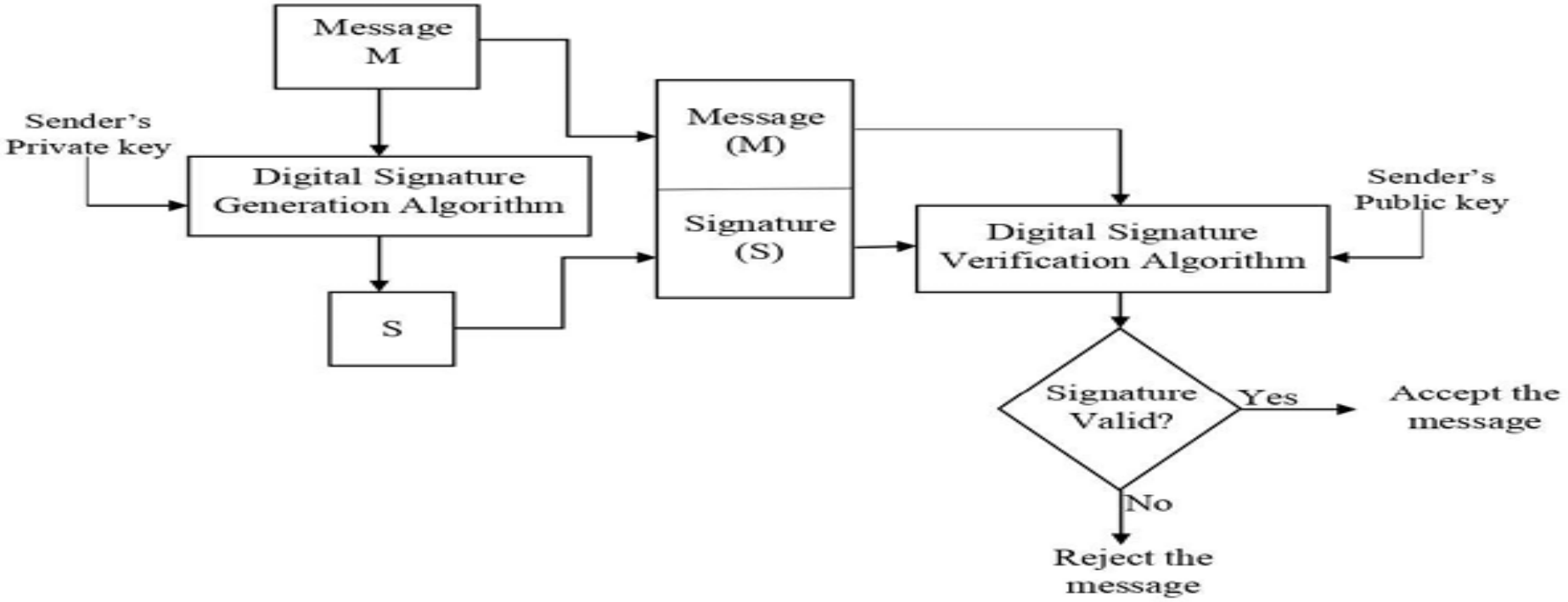


Fig: Generic model of Digital Signature

Digital Signature

In a digital signature process, the sender uses a signing algorithm to sign the message. The message and the signature are sent to receiver. The receiver receives the message and the signature and applies the verifying algorithm to the combination. If the result is true, the message is accepted otherwise it is rejected.

Direct and Arbitrated Digital Signature

- A ***direct digital signature*** involves only two parties, a sender and a receiver. It is assumed that the receiver knows the public key of the sender. A digital signature may be formed by encrypting the entire message with the sender's private key or by encrypting a hash code of the message with the sender's private key.
  - There is a threat of forged digital signatures in direct digital signature.

- An ***arbitrated digital signature*** involves three parties, a sender, a receiver and a trusted arbiter. Every signed message from a sender X to a receiver Y goes first to an arbiter A, who subjects the message and its signature to a number of tests to check its origin and content. The message is then dated and sent to Y with an indication that it has been verified to the satisfaction of the arbiter.
  -There is no threat of forged signatures because an independent arbiter verifies the entire process with due dating and time-stamping.

Digital Signature

- Bob can sign a message using digital signature generatic algorithm.
- The inputs to the generatic algorithm are the message and Bob private key.
- Any other user, say Alice, can verif the signature using a verificatic algorithm
- The inputs to the verificatic algorithm are the message, th signature, and Bob's public key.

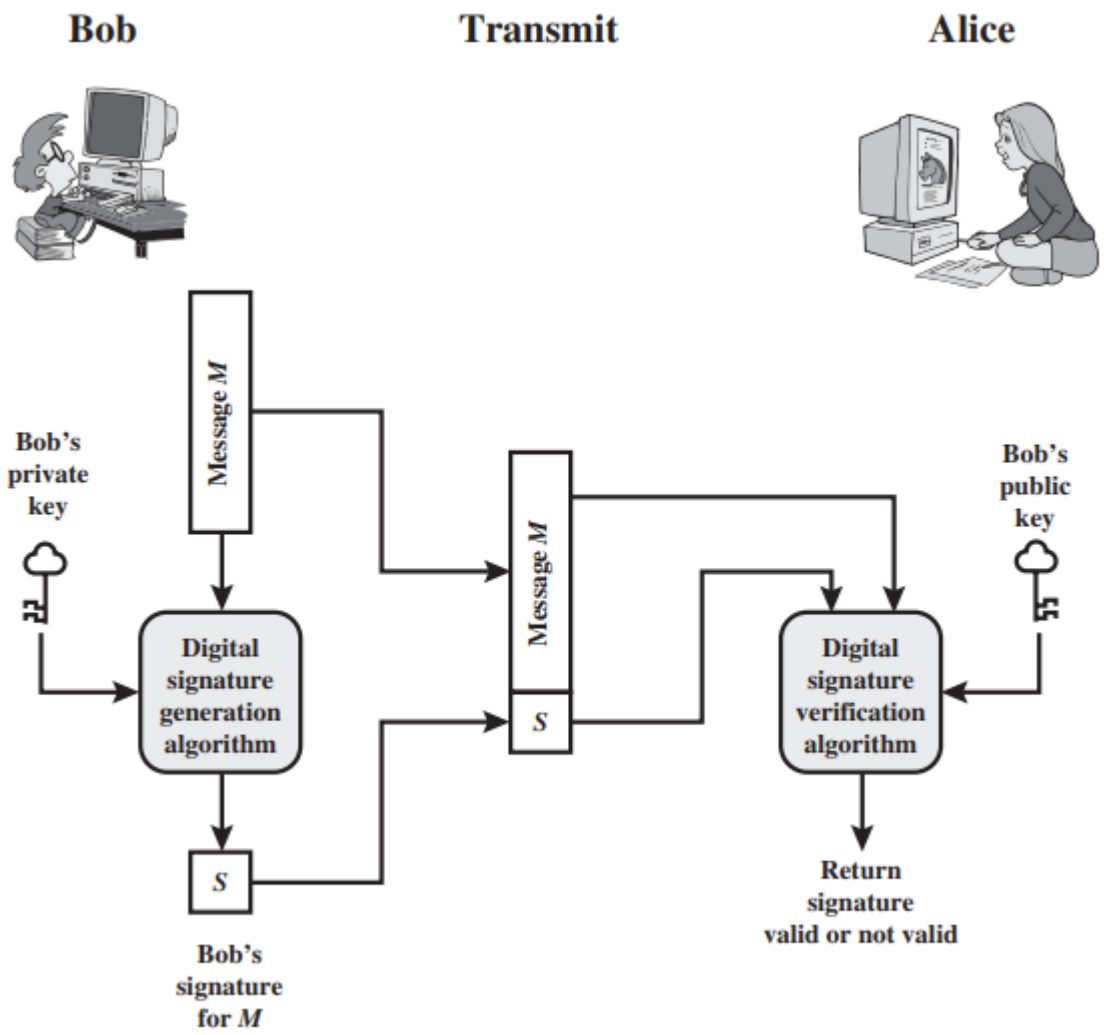


Figure 13.1 Generic Model of Digital Signature Process

Digital Signature

- A digital signature is more than just an authentication function:

- It can verify the author and the date and time of the signature.
- It can authenticate the contents at the time of the signature.
- It can be verifiable by third parties, to resolve disputes.

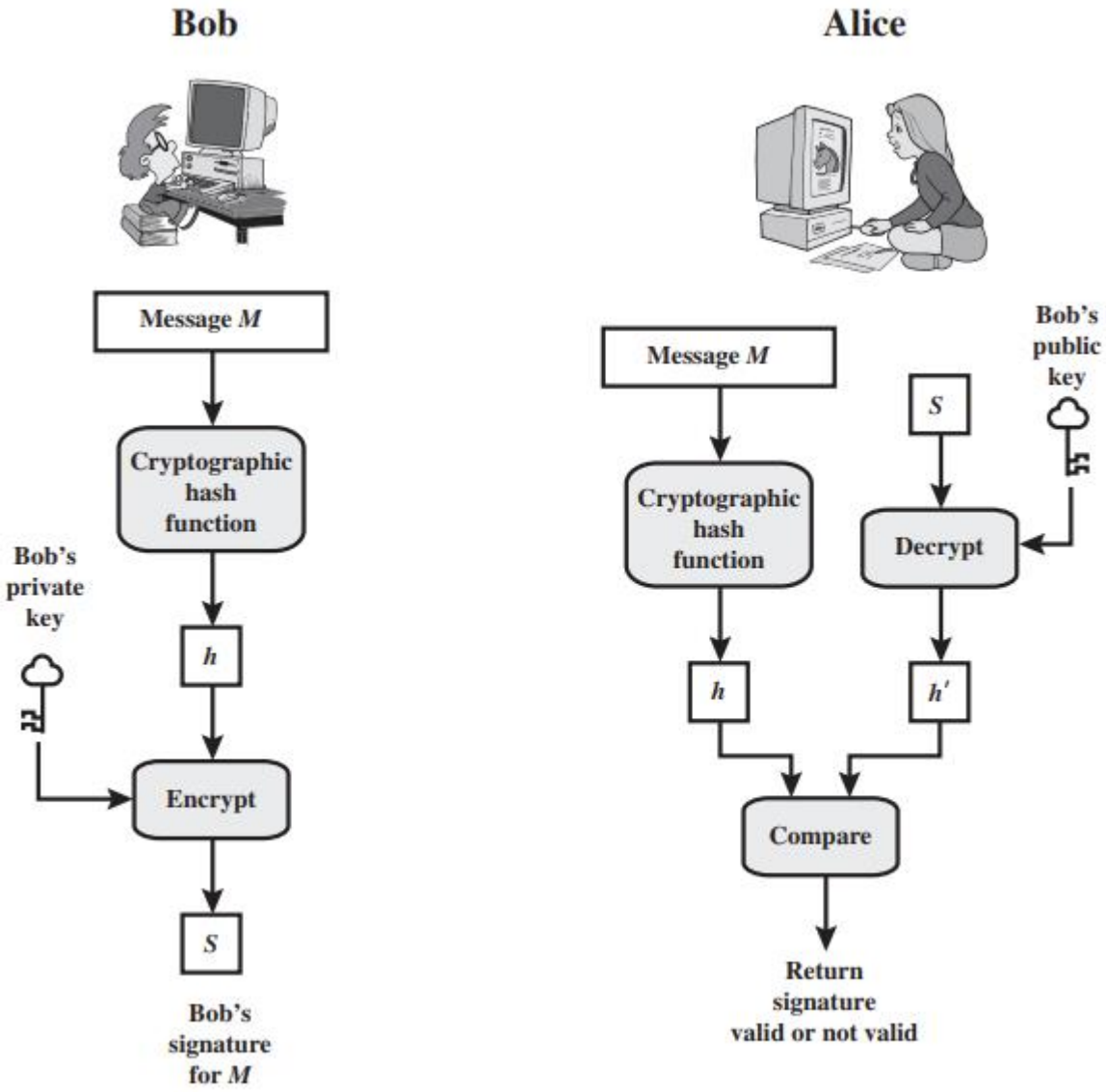


Figure 13.2 Simplified Depiction of Essential Elements of Digital Signature Process

Digital Signature Requirements

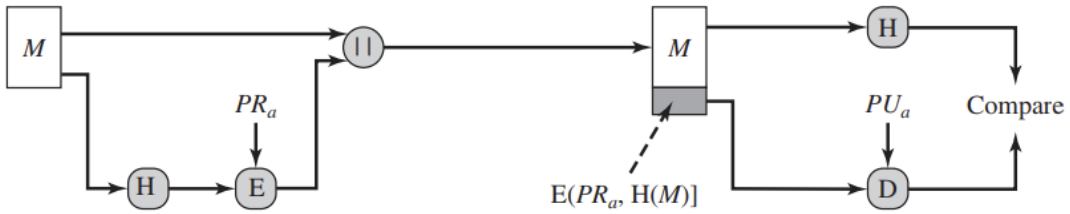
- The signature must be a bit pattern that depends on the message being signed.
- The signature must use some information unique to the sender to prevent both forgery and denial.
- It must be relatively easy to produce the digital signature.
- It must be relatively easy to recognize and verify the digital signature.
- It must be computationally infeasible to forge a digital signature, either by constructing a new message for an existing digital signature or by constructing a fraudulent digital signature for a given message.
- It must be practical to retain a copy of the digital signature in storage.

Digital Signature Standard (DSS)

- The National Institute of Standards and Technology (NIST) has published Federal Information Processing Standard FIPS 186, known as the Digital Signature Standard (DSS).
- The DSS was originally proposed in 1991 and revised in 1993 and 1996.
- In 2000, an expanded version of the standard was issued as FIPS 186-2, subsequently updated to FIPS 186-3 in 2009.
- The DSS makes use of the Secure Hash Algorithm (SHA) and presents a new digital signature technique, the Digital Signature Algorithm (DSA).

Approaches to Digital Signature

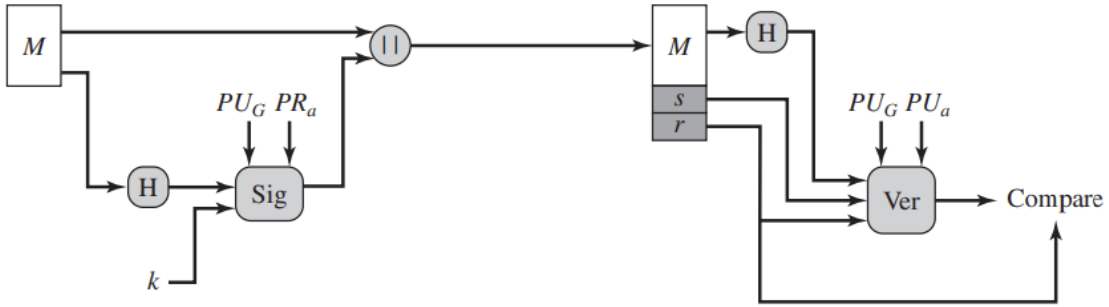
- **RSA approach:**
- A hash function produces a hash code of fixed length from the message. This hash code is encrypted using the sender's private key to form the signature.
- Both the message and the signature are then transmitted.
- The recipient takes the message and produces a hash code.
- The recipient also decrypts the signature using the sender's public key.
- If the calculated hash code matches the decrypted signature, the signature is accepted as valid.
- Because only the sender knows the private key, only the sender could have produced a valid signature.



(a) RSA approach

Approaches to Digital Signature

- **DSS approach:**
- A hash function is used as well.
- Signature function: The hash code is provided as input to a signature function along with a random number $\boldsymbol{k}$ generated for this particular signature, sender's private key $\boldsymbol{PR_a}$ and global public key $\boldsymbol{PU_G}$ (a set of parameters known to a group of communicating principals). We can consider this set to constitute a global public key. The result is a signature consisting of two components, labeled $\boldsymbol{s}$ and $\boldsymbol{r}$.
- At the receiving end, the hash code of the incoming message is generated.
- Verification function: The generated hash code, the signature ($\boldsymbol{s}$, $\boldsymbol{r}$), global public key $\boldsymbol{PU_G}$ and sender's public key $\boldsymbol{PR_a}$ are input to a verification function. The output of the verification function is a value that is equal to the received signature ($\boldsymbol{s}$, $\boldsymbol{r}$) if the signature is valid.



(b) DSS approach

Figure 13.3 Two Approaches to Digital Signatures

Digital Signatures	Hash Functions	Authentication codes (MACs)
They deal with 1024 bit minimum key length [3] and have not been patented or licensed yet.	Has functions mostly deals with 128 to 256 bit key. examples: MD5, RFC, SHA1, RACE	MACs are able to reduce message into short length. It is typically 32 or 64 bit long and key is 56 bit long. MAC= hash value+ Public Key
Digital signatures rely on public key cryptography; where verification key is public and signing key is referred as secret or private key.		MACs are based on secret key or private key.
As a prerequisite, parties did not need to agree on shared key before starting encryption.	As a prerequisite, parties did not need to agree on shared key before starting encryption.	As a prerequisite, a secret key needs to be shared therefore both parties need to agree on same key before starting the encryption through authentication code.
Digital signatures are slow in processing than hash function and symmetric encryption scheme.	Fast in processing speed.	Quite fast in processing.
Did not provide confidentiality.	Did not provide confidentiality.	Did not provide confidentially.
Digital signatures provide origin authentication.	They did not provide origin authentication.	They provide origin authentication.
Digital signatures provide integrity.	They provide integrity.	They provide integrity.